The Charlotte Operating System

An async-first, capability-based microkernel
for critical and performant software

2026-07-26

Contents

1 The Problem

1.1 The operating system is the bottleneck

Critical software has two requirements that pull in opposite directions: it must be **correct** (failures are expensive or dangerous), and it must be **performant** (latency and throughput matter). Modern systems try to satisfy both by piling layers onto an operating system whose fundamental abstractions were designed for timesharing, not for the workloads we run today.

The result is what we call the **retrofit tax**: the cost, in complexity and performance, of making an OS designed for synchronous, trust-everyone, monolithic execution behave as if it were asynchronous, least-privilege, and decomposed.

1.2 The retrofit tax, itemized

1.2.1 Async over blocking

Most operating systems expose synchronous system calls: `read()` blocks the calling thread until data arrives. When applications need concurrency without dedicating one thread per operation, they reach for `epoll`, `kqueue`, or `io_uring`. These are effective, but they are *retrofits*: the kernel internally still blocks on disk I/O, network I/O, and locks, while the completion mechanism translates those blocking events into readiness notifications for userspace.

This translation layer brings with it:

- Async-signal-safety problems — the kernel may deliver a completion while userspace holds a lock, requiring elaborate signal-handler discipline or a dedicated thread pool.
- Thread-pool overhead — many async runtimes spawn a pool of OS threads to convert blocking syscalls into futures, undoing the resource-efficiency argument for async in the first place.
- Two concurrency models — the application uses async/await; the kernel uses threads and blocking. The mismatch forces every boundary crossing to bridge two foreign paradigms.

1.2.2 Ambient authority

A conventional OS process has access to the filesystem namespace, the network, and other global resources by default. Security is subtractive: you start with everything and try to take things away (chroot, seccomp, namespaces). This is brittle because it requires the developer to enumerate what *should not* be accessible — an unbounded problem. In practice, most programs run with far more authority than they need, and exploits exploit that gap.

1.2.3 Hardware asynchrony disguised as software synchrony

Hardware is inherently asynchronous with respect to the CPU. A disk controller signals completion via an interrupt. A network card delivers packets on its own schedule. A timer fires after a programmed interval.

Yet the operating system interface presents these as synchronous calls: `read()` returns when the data is ready, blocking the calling thread in the meantime.

This is a leaky abstraction. The thread that blocks on I/O is an OS resource that cannot be used for anything else while it waits. High-concurrency servers compensate with thread pools; real-time systems compensate with carefully tuned polling loops. Neither is the natural solution.

1.2.4 Fault propagation across trust boundaries

A bug in a device driver can crash a monolithic kernel. A memory corruption in one application can, in a system without hardware-enforced isolation, corrupt another. Conventional microkernels solve this with address-space isolation, but at the cost of IPC overhead that often drives designers back toward monolithic architectures for performance.

The industry has settled on a compromise: run untrusted code in virtual machines; run trusted code in the kernel; hope the trusted code has no bugs. For critical systems, hope is a poor strategy.

1.3 Why Rust ownership is not enough

Rust's type system eliminates data races, use-after-free, and null-pointer dereferences at compile time. This is a transformative improvement for systems programming, but Rust programs still run on operating systems that do not share Rust's ownership discipline.

- A Rust program cannot protect itself from a C driver that corrupts kernel memory. Isolation must be enforced by the hardware (MMU, IOMMU), not by the language.
- A Rust `Mutex<Vec<T>` crossing a syscall boundary is still a shared-memory lock with all the contention, priority inversion, and deadlock risks that implies. The kernel does not know the lock exists.
- Rust's `async`/`await` compiles to a state machine driven by a userspace executor, but the executor must still interact with the kernel's blocking I/O model. The `Future` trait and the kernel's wait queue speak different languages.

The insight is: Rust eliminates *some* classes of bugs within a single address space. But the OS defines the rules for *between* address spaces — and if those rules are "any process can do anything until restricted," Rust's guarantees are local optimizations on a global problem.

1.4 What needs to change

We need an operating system whose foundational abstractions align with the demands of critical, performant software:

1. Asynchronous by construction: Operations that may wait for external progress should have submission/completion semantics, not blocking-call semantics. No subsystem should force a thread to dedicate itself to one operation.
2. Capability-based, not ambient, authority: A process should be able to perform an operation only if it holds a capability authorizing it. Capabilities are unforgeable and must be explicitly granted.
3. Isolation by default: Every protection domain should run in its own address space. Communication crosses a kernel-mediated boundary. A fault in one domain should not propagate to another.

4. Ownership that crosses boundaries: Data movement between domains should transfer ownership, not copy bytes. The kernel should move page mappings, so the sender loses access and the receiver gains it.
5. Bounded resource consumption: Every queue, table, and buffer should be bounded. Backpressure should propagate explicitly. No unbounded allocation should be a correctness requirement.
6. The OS and the language runtime should agree: The kernel's model of concurrency (asynchronous, completion-driven) should match the language's model of concurrency (futures, wakers). No translation layer should be needed.

CharlotteOS is an operating system built from these six principles. This manual explains the architecture, the programming model, and — most importantly — why the combination eliminates the retrofit tax and provides a strong foundation for software that must be both correct and fast.

2 Design Philosophy

CharlotteOS is the result of three independent projects converging on a shared thesis. This chapter explains what each contributes and how they fit together.

2.1 The combined thesis

> **Hardware is asynchronous; the operating system and the userspace runtime should be too.**

Three systems approach the same design space from different directions. Rather than collapsing their abstractions into one universal mechanism, the architecture composes them at the right boundaries.

2.1.1 CharlotteOS — the protection, scheduling, and interrupt substrate

CharlotteOS contributes the mechanisms that *must* be privileged:

- Address-space isolation through page tables and the MMU.
- Kernel scheduling of threads across logical processors.
- LP-local state and interrupt routing.
- Page-table manipulation and physical-memory management.
- MMIO and DMA authority delegation.
- Per-domain capability tables.
- Event observation and thread wakeup.
- Syscall dispatch and asynchronous completion delivery.

Its design rule is: **the kernel provides the minimum set of primitives from which everything else can be composed.** If a facility can be built in userspace without compromising isolation, it belongs in userspace.

2.1.2 Sitas — the shard-per-core execution discipline

Sitas contributes an execution discipline — a set of rules about how state is owned and mutated:

> Only the owning shard mutates shard-local state. Cross-shard interaction occurs through bounded typed messages carrying owned values.

Specifically, sitas provides:

- One executor per shard — cooperative task scheduling within a shard; preemption between shards.
- Futures as userspace state machines — every operation that may wait is a `Future`, driven by the shard's executor.
- Bounded backpressure — every queue has fixed capacity; senders must handle `WouldBlock`.

- Shard-local ownership — mutable state belongs to exactly one shard; no locks, no atomics for shard-internal data.
- Explicit placement — where a task runs is a deliberate choice, not an accident of scheduling.
- Typed command/reply APIs — what a shard receives is a specific Rust type, not a generic byte buffer.
- Structured shutdown and cancellation — tasks can be cancelled, and cancellation has defined ownership semantics.
- Observability through owned snapshots — diagnostics capture state by value, never by borrowed reference into runtime internals.

The sitas discipline is enforced by Rust's type system within a single protection domain. CharlotteOS extends it across domains through the kernel.

2.1.3 Xous — the isolated userspace service model

Xous (a microkernel OS for Precursor and Betrusted hardware) contributes the missing protection-domain service model. Its most valuable ideas for CharlotteOS are:

- Services are userspace servers: Drivers, filesystems, and protocol stacks run in their own protection domains, not in the kernel.
- Clients connect to servers and receive connection identifiers: A connection *is* authority; possessing one means you may communicate with that server.
- IPC is a kernel primitive: Communication is not a convention layered on shared memory; it is mediated and validated by the kernel.
- Scalar messages are distinct from memory-bearing messages: Small control-plane operations (opcodes, handles, status codes) travel in fixed-size messages; bulk data travels as memory-object attachments.
- Memory IPC distinguishes ownership transfer from borrowing: Move transfers ownership permanently. Borrow grants temporary access, revoked on reply or cancellation.
- A userspace name service maps human-readable names to opaque server identities. The kernel knows nothing about strings.

Xous demonstrates that a productive, secure operating system can be built entirely from userspace servers communicating through a small, well-defined kernel IPC primitive. CharlotteOS adopts this model and extends it with the sitas execution discipline and an async-native kernel.

2.2 How they fit together

The three systems do not collapse into one. They compose at well-defined boundaries:

Layer	Provided by
Shard-local async execution	sitas
Cross-domain protection and IPC	CharlotteOS kernel (Xous-inspired)
Completion delivery for kernel/device ops	CharlotteOS kernel
Service discovery and policy	Userspace (name service, supervisor)

A useful mental picture:

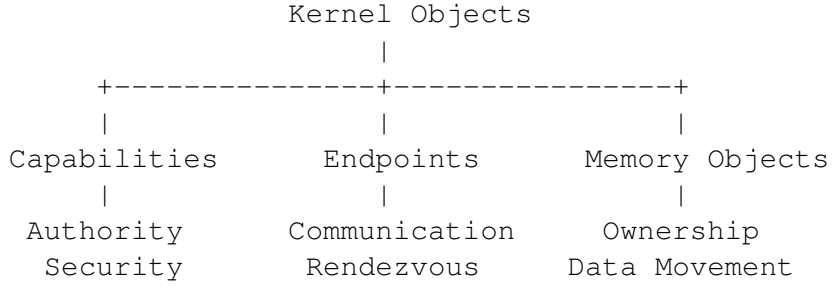
```
CharlotteOS kernel
    protection domains
    threads and LP scheduling
    capabilities and endpoint connections
    interrupt routing
    memory objects and page mappings
    operation submission and completion queues

Userspace service process
    one or more externally visible IPC endpoints
    protocol-specific request/reply handling
    one sitas executor per assigned shard
    shard-local state
    typed in-process commands
    driver or service implementation

Client process
    typed protocol library
    endpoint connection capabilities
    futures backed by IPC replies or completion records
    owned buffers whose transfer mode is explicit
```

2.3 The three kernel primitives

Before discussing IPC, drivers, scheduling, or asynchronous execution, the architecture must be understood in terms of three orthogonal kernel abstractions. These are the conceptual foundation of CharlotteOS:



Every other kernel facility — completion queues, reply tokens, device grants, DMA mappings — is a **composition** of these three primitives.

2.3.1 Capabilities: "May I?"

A capability answers one question: **"May I perform this operation?"** It represents authority, not work. Examples include endpoint capabilities, connection capabilities, memory-object capabilities, interrupt capabilities, DMA-domain capabilities, and timer capabilities.

2.3.2 Endpoints: "With whom?"

An endpoint answers: **"Who am I communicating with?"** It provides bounded rendezvous between protection domains. Endpoints know nothing about packet buffers, futures, DMA, or ownership. They carry requests and replies.

2.3.3 Memory Objects: "Where is the data, and who owns it?"

A memory object answers: **"Where is the data, and who currently owns it?"** It represents page-backed storage whose ownership can move independently of communication. Its responsibilities include page ownership, mappings, transfer modes, DMA state, and lifetime.

2.4 An explicit design rule

> **When adding a new subsystem, first ask whether it can be expressed using capabilities, endpoints, and memory objects. New kernel primitives should be introduced only when they cannot be naturally composed from the existing ones.**

This rule keeps the kernel small, the ABI auditable, and the trust model comprehensible. A developer reasoning about security needs to understand three concepts, not thirty.

2.5 What "asynchronous throughout" means

"Asynchronous throughout" is a design goal, which means:

1. Operations that may wait for external progress have submission/completion semantics.
2. No subsystem boundary forces the caller to dedicate a thread to waiting for one operation.
3. A userspace shard can wait on one aggregated kernel source (the CQ).
4. Completions are retained until observed.
5. Buffers and capabilities have explicit ownership during an operation.
6. Cancellation and teardown have defined state transitions.
7. Bounded queues preserve backpressure across layers.
8. Service implementations can suspend one task while continuing other work on the shard.

This is not the same as saying:

- Every syscall is asynchronous.
- Every call returns a newly allocated completion capability.
- No thread ever blocks.
- Every wake sends an IPI.
- All waitable objects have identical semantics.
- Userspace code is interrupted by arbitrary asynchronous callbacks.

2.5.1 Blocking is required, and desirable

When no sitas task is runnable, the shard's execution context should block in one kernel wait operation:

```
ready tasks exist
    -> poll tasks

no ready tasks
    -> wait on CQ / endpoint readiness / deadline

event arrives
    -> execution context becomes runnable
    -> drain events
    -> wake relevant futures
```

CharlotteOS's advantage is:

> **The shard parks directly on native completion and message state; no helper-thread pool is required merely to convert blocking kernel interfaces into futures.**

2.5.2 Immediate operations remain immediate

Operations that cannot meaningfully block — capability duplication, metadata queries, nonblocking CQ drain, closing a quiescent object — complete synchronously. The ABI does not force asynchronous ceremony onto inherently synchronous work.

2.5.3 No asynchronous userspace upcalls

Completions do not inject callbacks into userspace at arbitrary instructions. The path is:

```
hardware interrupt
    -> kernel records result or notification
    -> blocked execution context becomes runnable
    -> normal return to userspace
    -> sitas executor drains CQ/endpoints
    -> executor wakes relevant Rust tasks
```

Kernel preemption is allowed and arbitrary userspace task upcalls are not required. This keeps the userspace programming model cooperative, predictable, and auditable.

2.6 Why this matters for critical software

Every design decision in CharlotteOS traces back to one or more of the six principles from Chapter 1:

- Capabilities replace ambient authority: A compromised service cannot access resources it was never granted. The set of things a process can do is enumerable and auditable by construction.
- Endpoints make communication authenticated: A client that holds a connection capability has proven (at grant time) that it is authorized to communicate. The kernel enforces this on every message.
- Memory objects eliminate an entire category of bugs: Buffer over-reads, use-after-free, and double-free across protection boundaries are structurally impossible when ownership moves through the kernel and is enforced by page tables.
- Composition keeps the kernel small: A small kernel has a small attack surface. Most system functionality lives in userspace, where a bug is a service restart, not a kernel panic.
- The async model removes the retrofit tax: There is no translation layer between the kernel's concurrency model and the language's. This means fewer bugs in the translation, lower latency, and a simpler mental model.
- Bounded queues make resource consumption predictable: You cannot run a CharlotteOS system out of memory by sending too many messages. Backpressure propagates explicitly; the system degrades gracefully rather than failing catastrophically.

The following chapters build out this architecture in detail.

3 Vocabulary

This chapter establishes the terminology used throughout this manual. Many of these terms sound similar to concepts from other operating systems, but their CharlotteOS definitions are specific.

3.1 Protection domain

A **protection domain** is an address space and authority boundary. It owns:

- A capability table.
- Memory mappings.
- One or more execution contexts (threads).
- Resource accounting.
- Failure and teardown state.

A *process* is the concrete implementation of a protection domain, but architectural discussion prefers the semantic term.

3.2 Execution context

An **execution context** is a kernel-scheduled thread. It may have:

- LP affinity.
- Priority or scheduling class.
- A current protection domain.
- A wait state.
- An associated userspace executor.

3.3 Logical processor (LP)

A **logical processor** is CharlotteOS's representation of a schedulable hardware execution context — typically one CPU core (or one hyperthread).

An LP is **not** a shard (in the sitas sense). A typical mapping is:

```
one sitas shard
    <-> one LP-affine userspace thread
    <-> normally one logical CPU
```

The architecture retains the distinction so that CPU hotplug, oversubscription, migration, asymmetric CPUs, and multiple sharded processes remain possible.

3.4 Sitas shard

A **shard** is a userspace ownership and execution domain:

- One executor runs its tasks.
- At most one task at a time mutates shard-owned state.
- Cross-shard access occurs through typed messages carrying owned values.
- Placement is explicit but is not the shard's identity.

3.5 Capability

A **capability** is an unforgeable reference to a kernel object, carrying rights. It answers: "May I perform this operation?"

Capabilities are scoped to a protection domain's capability table. A process cannot fabricate a capability; it can only receive one through the bootstrap contract or through IPC.

3.6 Endpoint

An **endpoint** is a bounded server-side IPC receive object. It has:

- An owning protection domain.
- A queue capacity.
- A protocol/interface identity.
- Receive rights.
- Lifecycle and closure state.

3.7 Connection capability

A **connection capability** is a client-side authority to send or call an endpoint. Possession of a service name is not authority. A name service resolves a name and conditionally returns a connection capability.

3.8 Message

A **message** is a kernel-validated IPC envelope consisting of:

- Interface or protocol identifier.
- Opcode.
- Flags.
- Inline scalar words.
- Zero or more memory-object attachments.
- Zero or more delegated capability attachments.

- Optional reply authority.

The kernel understands the envelope structure, not arbitrary Rust types.

3.9 Reply token

A **reply token** is one-shot authority to complete a blocking or deferred call. It is:

- Created by the kernel.
- Bound to the caller's pending call.
- Consumable exactly once.
- Invalidated by cancellation, caller death, or endpoint teardown.
- Optionally delegable when explicitly allowed.

3.10 Resource capability

A **resource capability** authorizes operations on a kernel-managed object such as a memory object, endpoint, connection, interrupt, MMIO region, DMA domain, device, completion queue, or timer.

3.11 Operation and completion queue

An **operation** is submitted work whose progress may depend on the kernel, hardware, or a privileged service.

A **completion queue (CQ)** is the aggregated, per-shard or per-process destination for operation results. It is waitable (with `CQ_WAIT`) and has non-lossy terminal completion semantics.

3.12 Notification

A **notification** means: "State may have changed; inspect the corresponding object or queue." It may be coalesced. It is not itself a result.

3.13 Rust Waker

A Rust `Waker` means: "Poll this userspace future again." It is a userspace scheduling hint. It is **not** a kernel completion record and should not cross the ABI directly.

3.14 Distinctions that matter

Several concepts sound similar but must be kept separate. Confusing them leads to architectural mistakes.

Concept	Is	Is not
Capability	Unforgeable authority to act	An event, a file descriptor, a handle
Endpoint	Server-side bounded receive queue	A socket, a pipe, a channel
Completion	A result of an async operation	A Waker, a notification
Waker	A hint to repoll a future	A kernel completion record
Shard	A userspace ownership domain	An LP, a core, a thread
LP	A schedulable hardware context	A shard
Notification	“Inspect state”	A result, a data payload
IPC call	Cross-domain service invocation	A kernel operation submission

3.15 Quick reference: acronyms

ASID Address Space Identifier. 0 = kernel; >0 = user address space.
CQ Completion Queue. Shared-memory ring buffer for async operation results.
EL0 Exception Level 0 (AArch64). Userspace execution.
EL1 Exception Level 1 (AArch64). Kernel execution.
HHDM Higher Half Direct Mapping. Linear mapping of all physical memory into kernel virtual address space (provided by the Limine bootloader).
IPI Inter-Processor Interrupt. Signal from one LP to another.
LP Logical Processor. One schedulable hardware execution context.
MMU Memory Management Unit. Hardware that enforces page-table permissions.
IOMMU / SMMU I/O Memory Management Unit. Enforces DMA access permissions from devices, analogous to the MMU for CPU access.

With this vocabulary established, the following chapters build the architecture layer by layer.

4 Capabilities — Authority Without Ambiguity

A capability is the answer to one question: **"May I perform this operation?"** It is authority, not work. It is unforgeable, delegable, and attenuable. This chapter explains the capability model that secures every resource in CharlotteOS.

4.1 What a capability is

A capability is a per-address-space handle that names a kernel object and carries a rights mask. The kernel maintains a **capability table** for each protection domain. An entry in this table is the only way a userspace process can reference a kernel object.

Key properties:

- **Unforgeable.** A process cannot create a capability through computation. Capabilities enter a domain only through the bootstrap contract or through IPC delegation.
- **Scoped.** A capability is valid only in the domain that holds it. Capability index 3 in domain A refers to a different object (or nothing) than capability index 3 in domain B.
- **Righted.** A capability carries a rights mask specifying which operations are permitted. An endpoint capability might carry only `SEND | CALL` rights; it does not grant `RECEIVE`.
- **Revocable.** When a kernel object is destroyed, all capabilities referencing it become invalid. Subsequent use returns an error, not undefined behavior.

4.2 Object types and their rights

CharlotteOS defines a fixed set of kernel object types, each with its own rights model. A capability carries both a type tag and a rights mask.

Type	Example rights	What it authorizes
Endpoint	RECEIVE, MINT_CONNECTION	Server-side IPC receive queue
Connection	SEND, CALL, DELEGATE	Client-side authority to invoke a service
ReplyToken	COMPLETE, DELEGATE	One-shot completion of a pending call
CompletionQueue	SUBMIT, WAIT, DRAIN	Submission to / waiting on a CQ
MemoryObject	MAP, UNMAP, MOVE, LEND, COPY	Page-backed memory ownership and access
Interrupt	ACK, MASK, BIND_TO_CQ	Hardware interrupt handling
MmioRegion	MAP_DEVICE	Device register access (MMIO)
DmaDomain	PIN, UNPIN	DMA buffer management
Device	RESET, CONFIGURE	Device lifecycle management
Timer	ARM, CANCEL, BIND_TO_CQ	Timer operations

4.3 Delegation and attenuation

Capabilities can be **delegated** to another protection domain through IPC. When delegating, the sender may **attenuate** the rights: the receiver gets a subset of the rights the sender held. A service manager that holds a connection with `SEND | CALL | DELEGATE` rights can delegate a client-facing connection with only `SEND | CALL`, keeping `DELEGATE` for itself.

Delegation happens in the kernel, mediated by the IPC path. The sender specifies the target domain, the capability to delegate, and the reduced rights mask. The kernel validates that the sender actually holds the capability with at least the rights being transferred.

4.4 Generation-safe lookup

Capability tables use generation counters to prevent use-after-free through stale indices. When a capability is revoked, the table slot is freed and its generation is incremented. Any attempt to use an old index with a stale generation returns `InvalidCapability` rather than accessing a different object that happened to be allocated at the same index.

4.5 Bootstrap — how capabilities enter a domain

A newly created protection domain starts with an empty capability table. It receives its initial capabilities through the **config-page contract**: a mapped page that the supervisor populates with typed entries before the domain begins execution. The bootstrap contract delivers at minimum:

- A connection to the name service (for service lookup).
- For driver domains: device capabilities (MmioRegion, Interrupt).
- For service domains: an endpoint on which to listen.
- A default CQ handle for submission and waiting.

After bootstrap, additional capabilities arrive through IPC: the name service delegates service connections; the supervisor delegates device grants; other services delegate whatever the protocol authorizes.

4.6 Why capabilities matter for critical software

1. Least privilege is architectural, not advisory: A process starts with nothing. Every capability it holds was explicitly granted by a trusted component (the supervisor, the name service, or another service under a protocol). There is no ambient authority to strip away; the default is zero.
2. The attack surface is enumerable: To determine what a process can do, inspect its capability table. There is no hidden state, no global filesystem, no ambient network access. Auditing is bounded.
3. Delegation is mediated: A process cannot grant authority it does not hold; it cannot grant rights beyond what it possesses; and every delegation crosses the kernel, which enforces both constraints.
4. Revocation is complete: When a service crashes and its domain is torn down, every capability to every connection it held is invalidated atomically. There is no lingering authority.

5. Capability tables are bounded: A process cannot create unlimited capabilities to exhaust kernel memory. Submission backpressure (`WouldBlock`) applies when a table is full, forcing the application to drain completions before submitting more work.

The capability model is the foundation on which CharlotteOS builds its security guarantees. Without capabilities, endpoints and memory objects would be anonymous resources accessible by index or address — just a more elaborate version of the ambient-authority model this architecture rejects.

5 Endpoints — Communication as a First-Class Primitive

An endpoint answers one question: **"Who am I communicating with?"** It is the mechanism by which protection domains invoke each other. This chapter explains the endpoint IPC model — CharlotteOS's primary communication primitive.

5.1 Server and connection model

An **endpoint** is a bounded server-side IPC receive object. A service creates one and receives messages on it. A **connection capability** is a client-side authority to send to that endpoint. Connections are minted from the endpoint (typically by the name service) and delegated to clients.

```
Service creates endpoint:   endpoint_create(interface="ns", capacity=16)
Name service mints conn:    connection_mint(endpoint, SEND | CALL)
Client receives conn:       name_service.lookup("ns") -> ConnectionCap
```

Ordinary clients do not mint arbitrary connections. The name service or a policy service normally delegates them after an access check.

5.2 Message classes

CharlotteOS supports two fundamentally different kinds of messages, and the ABI keeps them distinct:

5.2.1 Scalar messages

A scalar message carries an opcode, one 64-bit argument word, and optionally a reply token. It fits in registers. It is suitable for:

- Opcodes and request types.
- Capability handles.
- Status values and queue indices.
- Compact control-plane operations where the payload is a single word.

Scalar messages are **fast**: they require no memory-object allocation, no page-table manipulation, and no TLB invalidation. The entire send/receive cycle is a handful of register copies.

5.2.2 Memory-bearing messages

A message can carry one or more **memory-object attachments** with explicit transfer modes (see Chapter 6). These are suitable for:

- Structured requests and responses.
- Strings, names, and configuration structures.
- Packet payloads and block I/O buffers.
- Larger replies that exceed register width.

The kernel validates each attachment: the sender must hold the referenced memory object with sufficient rights for the declared transfer mode. The receiver gets a new capability to the memory (or a temporary mapping, for borrows).

5.3 Message envelope

The kernel validates a fixed-width message envelope. It does not understand Rust enums, `serde` formats, or protocol-specific types:

```
struct MessageHeader {
    interface: u128,       // protocol identity
    version: u32,
    opcode: u32,
    flags: u32,
    inline_len: u16,
    segment_count: u16,
    capability_count: u16,
    reserved: u16,
    user_tag: u64,
}
```

Constraints on the ABI:

- Fixed-width fields — no `usize`, no pointer-sized values.
- Explicit alignment — no compiler-dependent layout.
- Checked offsets and lengths — the kernel rejects out-of-bounds.
- Explicit protocol versioning — forward compatibility is negotiated.
- Maximum attachment counts — bounded kernel work per message.
- No arbitrary userspace pointers as durable message identity.

5.4 Call and reply

The most common IPC pattern is **call/reply**: a client sends a request and expects a response.

5.4.1 Synchronous-looking futures

At the Rust level, a client writes:

```
let response = network_service.connect(address).await?;
```

Underneath, the runtime:

1. Sends a call message through the connection capability.
2. The kernel creates a one-shot **reply token** bound to this call.
3. The caller task becomes pending (the `Future` returns `Poll::Pending`).
4. The caller shard continues running other tasks — no thread is dedicated to this request.
5. The server receives the message on its endpoint.
6. The server replies (immediately or later).
7. Reply arrival wakes the client future, which returns `Poll::Ready(response)`.

No client OS thread is dedicated to the request. The shard moves on to other work.

5.4.2 Deferred replies

A server may retain a reply token and complete it later — while continuing to serve other requests on the same shard. This is essential for device-driven operations:

```
async fn handle_read(pending: PendingRequest) {
    // Submit a DMA read, retaining the reply token.
    // Continue serving other clients while the read is in flight.
    let buffer = device.read(address, len).await?;
    pending.reply_with_buffer(buffer);
}
```

The server is never blocked waiting for one slow operation. The shard's executor polls other tasks while the read completes.

5.4.3 Reply token semantics

A reply token is:

- **Created by the kernel** when a call message is enqueued.
- **Bound to the pending call** — it completes exactly that caller's future.
- **Consumable exactly once** — a second reply attempt returns an error.
- **Invalidated by cancellation** — if the caller times out or aborts, the reply token becomes inert.
- **Invalidated by endpoint teardown** — if the server dies, all outstanding reply tokens are invalidated, and all waiting callers receive `ServiceRestarted`.
- **Optionally delegable** — a server may forward a reply token to another service, allowing a chain of services to contribute to a single reply.

5.5 Connection delegation

A connection capability can be delegated to another domain with attenuated rights. This is the mechanism by which the userspace name service returns service-level connections:

```
// Name service:
let full_connection = name_service.lookup("driver.nic.v1")?;
// full_connection has SEND | CALL | DELEGATE rights

// Delegate to a client with reduced rights:
connection_delegate(full_connection, client_domain, SEND | CALL)?;
// client_domain now holds a connection with SEND | CALL only
```

The kernel mediates. The name service never sees raw endpoint identities; it holds connection capabilities, which it re-delegates. Each delegation step can only reduce rights.

5.6 Service identity and discovery

The kernel uses opaque endpoint identities. A userspace **name service** maps human-readable names to re-delegable connection capabilities:

- **Registration:** a service registers as `"driver.nic.v1"` with a specific endpoint. The name service stores the endpoint's connection.
- **Generation tracking:** each registration bumps a generation counter. Stale client connections bound to the previous generation fail with `ServiceRestarted`.
- **Access-key gating:** a registration may specify a non-zero access key. Lookups without the matching key are denied — policy enforced by a userspace service, not the kernel.
- **Lookup:** a client asks for `"driver.nic.v1"` and receives a connection capability (or an error). It never receives an address, a port number, or a raw handle.

The name service runs in userspace. The kernel provides the endpoint and connection primitives; userspace provides the naming policy.

5.7 Two kernel data paths, not one

A critical architectural distinction: endpoint IPC is for **cross-domain service calls**. Completion queues are for **kernel and device operations**. They are not the same mechanism.

Endpoint IPC path	Completion queue path
Application → network service	Timer expiry
Network service → NIC driver	DMA completion
Application → filesystem service	IRQ-derived device completion
Service → name service	Waiting for thread/process exit

A userspace driver may compose both: receive an endpoint message (data path), then submit a hardware operation and await a CQ entry (control path). The endpoint message and the hardware completion are related *in the service's logic*, but they are different kernel abstractions with different semantics.

5.8 Why endpoints matter for critical software

1. **Communication is authenticated by construction.** A client that holds a connection capability was authorized (at grant time, by the name service or policy service) to communicate with the target. Every message is implicitly authenticated — not by an IP address, not by a UID, but by possession of a kernel-mediated capability.
2. **Scalar and memory messages are distinct.** The kernel does not copy arbitrary data between address spaces by default. Small control messages stay small and fast. Bulk data moves as memory objects (Chapter 6) with explicit ownership semantics — no accidental copies, no buffer over-reads.
3. **Deferred replies enable efficient concurrency.** A server is never forced to block waiting for one slow client. It retains reply tokens for in-flight work and continues serving other requests. The shard's executor multiplexes naturally.
4. **Service restart is reliable.** Stale connections fail deterministically with typed errors (`EndpointClosed`, `ServiceRestarted`). A restarted service cannot accidentally inherit outstanding reply tokens from a previous instance.
5. **Name service is policy, not mechanism.** The kernel provides endpoints and connections. Userspace decides who can register what name, who can look up what service, and what access keys are required. The kernel remains small; policy is replaceable without a kernel rebuild.

6 Memory Objects — Ownership That Crosses Boundaries

A memory object answers: **"Where is the data, and who currently owns it?"** It is the mechanism by which data crosses protection-domain boundaries without copying bytes. This chapter explains CharlotteOS's memory-object model.

6.1 The problem with copying

In a conventional OS, moving data between processes typically means copying: the kernel reads from the sender's address space and writes to the receiver's. For large payloads (packet buffers, file blocks, IPC messages), this copy dominates the cost of the operation.

The alternative — shared memory — eliminates the copy but eliminates the safety. Two processes with writable mappings to the same physical pages can corrupt each other's data. Synchronization is advisory; enforcement is absent.

CharlotteOS adopts a third approach: **ownership transfer through page-table manipulation.** When a memory object moves from one domain to another, the kernel changes the page-table mappings. The sender loses access; the receiver gains it. No bytes are copied. The MMU enforces the ownership.

6.2 First-class memory objects

A memory object is a dedicated kernel object representing page-backed storage with explicit ownership and capability semantics. The kernel manages:

- **Physical frames** — the backing storage.
- **Mappings** — which domains have virtual addresses pointing to the pages, and with what permissions.
- **Ownership** — which domain currently holds the master capability.
- **Transfer state** — whether the object is idle, in transit, borrowed, or undergoing DMA.
- **Lifetime** — when the object is freed, all mappings are revoked and all capabilities to it are invalidated.

Userspace never exchanges raw pointers across protection domains. Instead, IPC messages attach **memory-object capabilities** with declared transfer modes.

6.3 Four transfer modes

CharlotteOS defines four transfer modes, enforced by the MMU:

6.3.1 Copy

The receiver gets a byte-for-byte duplicate. The sender retains the original, unmodified.

- **Use when:** the data is small, the sender needs to keep using it, or the receiver is not trusted with the original.
- **Kernel work:** allocate new physical frames, copy the contents, map them into the receiver's address space.
- **Semantics:** simplest; both domains can read their respective copies independently.

6.3.2 Move

Ownership transfers to the receiver. The sender loses all mappings and all capability rights. The receiver becomes the new owner, responsible for subsequent transfer or destruction.

- **Use when:** the sender is finished with the data and wants to hand it off without copying.
- **Kernel work:** validate sender ownership, prepare receiver mappings, remove sender mappings, perform TLB invalidation, commit ownership, publish the IPC message.
- **Semantics:** physical pages never move. Mappings change. After the move, any attempt by the sender to access the (now unmapped) virtual addresses faults.

6.3.3 BorrowRead

The receiver gets a temporary read-only mapping. The sender retains ownership. The borrow is revoked when the receiver replies (or the call is cancelled, or the server dies).

- **Use when:** the receiver needs to inspect data but must not modify it.
- **Kernel work:** map the pages into the receiver's address space with read-only permissions. Track the borrow so it can be revoked.
- **Semantics:** writable aliases in the receiver are forbidden by the page-table permissions. The receiver cannot write even through unsafe code. Revocation at reply time unmap the pages.

6.3.4 BorrowWrite

The receiver gets a temporary exclusive writable mapping. The sender's access is revoked for the duration of the borrow. When the receiver replies, sender access is restored.

- **Use when:** the receiver needs to write into a buffer the sender will later read (e.g., a read-into-user-buffer pattern). This is *mutable lending*.
- **Kernel work:** map the pages writable into the receiver, temporarily revoke (unmap or read-protect) sender access, track the borrow for reply-time revocation.
- **Semantics:** the hardware enforces mutual exclusion. The sender cannot access the pages while they are lended out. No amount of unsafe Rust can violate this — the MMU will fault.

6.4 Why hardware enforcement matters

The transfer modes are enforced by page tables, not by Rust's type system:

- A C program running in a protection domain cannot violate the constraints any more than a Rust program can.
- A buffer overrun cannot read pages that are not mapped.
- An unsafe block cannot write to a `BorrowRead` buffer.
- A use-after-drop cannot resurrect a moved memory object.

Rust's ownership complements the kernel enforcement — it catches errors within a domain at compile time. But the kernel is the ultimate guarantor for cross-domain safety, so non-Rust or buggy-Rust processes are still contained by page tables and capability checks.

6.5 Control plane versus data plane

Endpoint IPC belongs to the **control plane**: it configures connections, carries opcodes, sets up device queues.

Memory objects belong to the **data plane**: they carry the actual bytes (packets, blocks, buffers, strings).

For networking specifically:

- **Control plane:** endpoint IPC configures NIC queues, sets MTU, registers buffer pools.
- **Data plane:** descriptor rings coordinate producer/consumer state; packet payloads move as memory objects.

This separation avoids turning every packet into a copied IPC payload while preserving strict ownership.

6.6 DMA as a third ownership dimension

When a device performs DMA, a memory object enters a third ownership state:

- **CPU ownership:** the CPU reads and writes the pages normally.
- **DMA ownership:** the device is reading or writing the pages; the CPU must not access them.
- **Rust/API ownership:** which domain holds the capability.

The kernel tracks DMA state per memory object. A domain that submits a memory object for DMA loses CPU access until the operation completes. The IOMMU/SMMU extends capability enforcement to DMA transactions: a device can only access memory objects explicitly pinned into its DMA domain.

6.7 Why memory objects matter for critical software

1. **Zero-copy without loss of safety.** Pages move by changing mappings, not by copying bytes. This is faster than a copy and safer than shared memory.
2. **Use-after-free is structurally impossible across boundaries.** When a memory object is moved, the sender's mappings are removed. Any access faults. When a borrow is revoked, the borrower's mappings are removed. There is no window for a stale reference.

3. **Borrows have defined lifetimes.** A borrow is always tied to an IPC call. When the call completes (reply, cancellation, timeout, or server death), the borrow ends. There is no "I forgot to return it."
4. **DMA is tracked explicitly.** The kernel knows whether a memory object is currently owned by the CPU or a device. It will not allow one to be freed while the other is using it. This prevents the class of bugs where a driver frees a DMA buffer before the device finishes writing.
5. **Hardware enforcement, not language enforcement.** The page tables are the ultimate arbiter. No amount of unsafe code, C FFI, or compiler bug can access pages that are not mapped. The trust boundary is architectural.

7 Completion Queues — Asynchronous Operations Without Loss

A completion queue is the mechanism by which the kernel delivers results of asynchronous operations to userspace. This chapter explains the CQ model, the operation lifecycle, and the non-lossy invariant that makes CharlotteOS suitable for critical software.

7.1 When to use a completion queue

Use submission/completion when the operation belongs to the **kernel or a hardware-facing mechanism**, not to another userspace service:

- Timer expiry.
- DMA completion (disk read finished, packet transmitted).
- IRQ-derived device completion (data available on UART).
- Asynchronous page operations.
- Waiting for thread or process exit.
- Kernel-managed asynchronous object work.

Use endpoint IPC (Chapter 5) when one protection domain invokes another. Confusing these two paths leads to architectural mistakes: a service call is not a kernel operation, and a reply token is not a completion record.

7.2 The CQ ring — a shared-memory completion buffer

A completion queue is a **shared-memory ring buffer** between the kernel (producer) and userspace (consumer). A single 4 KiB page contains a header and a circular array of fixed-size entries.

Each entry is 32 bytes:

```
struct CompletionEntry {
    operation: u64,    // operation identifier (e.g., timer id)
    user_data: u64,    // opaque cookie set by the submitter
    status: u32,       // completion status code
    flags: u32,        // per-entry flags
    result: i64,       // operation result (e.g., bytes read)
    returned_capability: u64, // returned resource capability id
}
```

The ring is mapped into the user address space with read-only permissions for the consumer (userspace) and read-write for the producer (kernel). This means:

- **Userspace drains entries without syscalls.** Advancing the consumer tail is a store to a shared page; reading entries is a load. No kernel crossing needed for the common fast path.

- **The kernel writes entries with a release fence.** The consumer sees coherent data without additional synchronization.
- **Overflow is detected, not hidden.** An overflow counter in the header lets userspace detect when entries were produced faster than consumed.

7.3 The non-lossy invariant

> **A terminal operation result must never be silently lost because the userspace CQ ring is full.**

This is an architectural invariant, so when the ring is full, the kernel has several options:

- **Retain overflow in a per-domain kernel backlog.** Entries that cannot fit in the ring wait in a kernel-side buffer. When userspace drains the ring, the backlog drains into it.
- **Stop accepting new submissions.** The kernel may return `WouldBlock` on `submit`, applying backpressure at the submission point rather than at the completion point.
- **Propagate `WouldBlock`.** The CQ's wait operation can indicate that the ring is full and must be drained.

The ring's overflow counter is **pressure telemetry** indicating to the consumer that "You are falling behind."

In a critical system, a lost completion for a disk write or a timer expiry is a correctness bug with potentially severe consequences. CharlotteOS treats each completion as a datum that must be delivered at least once.

7.4 Per-shard CQ partitioning

Each address space owns a set of completion queues, keyed by `CqId`. Queue 0 is the default. For a service with multiple sitas shards, **per-shard CQ rings** enable targeted wakes:

- Shard A's executor waits on CQ 0.
- Shard B's executor waits on CQ 1.
- A completion destined for Shard A is posted to CQ 0.
- Only Shard A's executor is woken; Shard B is undisturbed.

This prevents cross-shard completion stealing (a pathological behavior where one busy shard drains entries intended for another) and keeps wakeup latency proportional to the number of entries, not the number of shards.

7.5 Operation lifecycle

Every asynchronous operation transitions through a defined state machine:

```
Created
    -> Submitted
```

```
-> Accepted | Rejected
-> InFlight
-> Completed | Failed | Cancelled
-> ResultObserved
-> ResourcesReclaimed
```

Each transition is explicit and observable:

Created The operation descriptor exists in userspace but has not been submitted to the kernel.
Submitted `submit()` has been called. The kernel has the operation record but has not yet validated it.
Accepted / Rejected The kernel has validated the operation. It may be rejected immediately (invalid parameters, insufficient rights, queue full) with a synchronous error.
InFlight The operation is being processed. The kernel or device owns any associated buffers.
Completed / Failed / Cancelled A terminal state. The result is posted to the CQ. Buffer ownership returns to userspace.
ResultObserved The consumer has read the CQ entry.
ResourcesReclaimed The operation's capability-table slot has been freed (via `close`).

7.6 Waiting on a CQ — `CQ_WAIT`

When a shard's executor has no ready tasks, it calls `CQ_WAIT` on its assigned queue. The kernel blocks the calling thread until one of three things happens:

1. A completion entry is posted to the CQ.
2. An explicit `CQ_WAKE` arrives (from another shard or the kernel).
3. A deadline elapses (timeout variant: `CQ_WAIT_TIMEOUT`).

The wait is race-free. The sequence is:

1. Inspect the CQ ring: if completions are pending, return immediately.
2. Register a waiter against the CQ's generation counter.
3. Re-inspect the CQ ring: if completions arrived during registration, unregister and return.
4. Block the thread. It will be woken when the generation counter changes (a new entry is posted or a `CQ_WAKE` bumps it).

This is the **single wait point** for the entire shard. Three event sources converge on it:

- Kernel completions (CQ entries from `submit` operations).
- Endpoint readiness (a coalesced wake when messages arrive on a bound endpoint).
- Device interrupts (a coalesced wake from a bound interrupt object).

The shard does not need separate wait operations for timers, IPC messages, and device events. One aggregated wait covers all of them.

7.7 LP affinity for CQ waits

A thread that blocks in `CQ_WAIT` is assigned an affinity LP. When woken, the scheduler returns it to the same LP rather than scanning for the least-loaded one. This has three benefits:

1. **Timer events stay on the same LP's queue,** eliminating cross-LP migration races (observer-not-found when a completion is posted on a different LP than the one where the waiter was registered).
2. **Cache warmth is preserved.** The waking thread finds its data in the LP's local caches.
3. **Shard-to-LP affinity is maintained.** The sitas shard that blocked on CQ 0 on LP 2 wakes up on LP 2, where its shard-local state lives.

7.8 Operation IDs versus per-operation capabilities

An earlier experimental design returned a `CompletionCap` for every submission. The final ABI uses **operation IDs** for the common path:

- `submit()` returns an `OperationId` (a lightweight identifier).
- The result arrives in the CQ with the same operation ID.
- No capability-table slot is consumed for common operations.
- A first-class operation capability is created only when independent waiting, cancellation, delegation, inspection, or policy requires it.

This avoids capability-table pressure for high-rate operations (timer events, network packet completions) while preserving the capability model for operations that need first-class authority.

7.9 Cancellation and buffer ownership

When a userspace task drops a completion handle or explicitly cancels an in-flight operation:

1. The kernel marks the operation as `Cancelling` but **retains any transferred buffer.** The buffer may still be the target of DMA or kernel access.
2. A terminal `Cancelled` completion is eventually posted to the CQ, returning buffer ownership to userspace.
3. On domain teardown (process exit), outstanding buffers may be leaked rather than freed while the kernel or a device may still touch them. Reclamation goes through explicit operation and device teardown.

This contract ensures that a buffer is never freed while still reachable by the kernel or a device. The safe Rust wrapper around an owned buffer consumes the value on submission and returns it on completion; there is no path to use-after-free even through cancellation.

7.10 Why completion queues matter for critical software

1. **Non-lossy completions are a correctness guarantee.** A disk-write confirmation, a timer expiry, or a sensor reading is a fact and losing it is a bug. CharlotteOS guarantees delivery.

2. **The single aggregated wait eliminates a class of race conditions.** A shard that must wait on "timer or IPC message or device interrupt" in a conventional OS must manage three separate wait objects, with potential lost-wake races between checking one and arming another. One CQ means one atomic check-and-arm sequence.
3. **Per-shard CQ rings enable predictable latency.** No shard wakes another shard's executor to deliver a completion. Wakes are targeted, bounded, and proportional to the actual work.
4. **The CQ ring is zero-syscall in the common case.** The fast path — drain entries, process results, loop — requires no kernel transitions. Only when the ring is empty and the shard must block does a syscall occur.
5. **Cancellation is defined, not best-effort.** Every cancellation path ends in a terminal completion. There is no "the buffer might or might not be freed" ambiguity. The ownership contract is explicit and testable.

8 Shards and Logical Processors

CharlotteOS separates two concepts that are often conflated: the hardware execution context (logical processor, or LP) and the userspace ownership domain (shard). This chapter explains both and the relationship between them.

8.1 The logical processor (LP) — the hardware unit of scheduling

An LP is CharlotteOS's representation of a schedulable hardware execution context. In most configurations, one LP equals one CPU core (or one hyperthread). The kernel shards its own state per-LP, meaning each LP has:

- Its own run queue of ready threads.
- Its own timer queue and quantum timer.
- Its own scheduler instance (e.g., `RoundRobin`).
- Its own IPI command queues for receiving work from other LPs.
- Per-LP data structures accessible through `PerLp<T>`.

An LP is a **place**: threads are pinned to LP at creation and (by default) stay there. Migration is opt-in and restricted.

8.2 The shard — the userspace unit of ownership

A shard is a userspace ownership and execution domain. The rules of the shard are the sitas discipline:

1. **Only the owning shard mutates shard-local state.** There is no lock on shard-local data because there is no concurrent access. The executor runs one task at a time on the shard.
2. **Cross-shard interaction goes through bounded typed messages.** A value that moves from shard A to shard B is sent through a typed channel (`ShardSender<M>`). The sender loses the value; the receiver gains it.
3. **Placement is explicit.** Code that needs to be shard-local is accessed through `ShardLocal<T>`, which verifies at runtime that the calling thread belongs to the owning LP.
4. **Observability returns owned snapshots.** Diagnostics and introspection capture state by value. No borrowed reference into shard internals escapes the shard.

8.3 The mapping: one shard ↔ one LP

In the typical configuration:

```
one sitas shard
    <-> one LP-affine userspace thread
    <-> normally one logical CPU
```

The userspace thread is pinned to its LP. Its executor runs only on that LP. Its shard-local state (`ShardLocal<T>`) is valid only when the owning LP's thread is executing.

The architecture retains the distinction (shard $\neq$ LP) so that more complex configurations remain possible:

- **Multiple sharded processes:** one LP might host shards from multiple protection domains, time-sliced by the kernel scheduler.
- **CPU hotplug:** an LP can be taken offline; its shards can be migrated (when migration-safe) or torn down.
- **Oversubscription:** a test can run N shards on $M < N$ LPs to exercise migration and scheduling paths.

8.4 `PerLp<T>` — interrupt-safe shared data

For kernel data that must be accessible from multiple LPs (including from interrupt context), CharlotteOS uses `PerLp<T>`: a boxed slice of per-LP cells, each wrapped in a lock. Access is:

```
per_lp_data.with(|lp_data| { lp_data.do_something(); });
```

The `with` closure acquires the lock for the current LP, calls the closure, and releases the lock. This is interrupt-safe (the lock masks interrupts during the critical section on architectures where that is necessary).

Use `PerLp<T>` for data that is:

- Accessed from interrupt handlers (timers, device IRQs).
- Modified by an IPI from another LP.
- Part of the scheduler, memory allocator, or other kernel subsystem that must be LP-safe.

8.5 `ShardLocal<T>` — lock-free single-owner data

For kernel data that is accessed only from one LP and never from interrupt context, CharlotteOS provides `ShardLocal<T>`: a lock-free per-LP container using `UnsafeCell<T>` with runtime owner-check and borrow-flag guard. Access is:

```
shard_local.with(|val| { *val = 42; });
```

The `with` closure:

1. Checks that the calling thread belongs to the owning LP (panics if not).
2. Checks that the borrow flag is clear (panics on re-entrant access).
3. Sets the borrow flag.
4. Calls the closure with `&mut T`.
5. Clears the borrow flag.

The closure must not:

- Store the `&mut T` reference anywhere (it must not escape).
- Send it to another thread or LP.
- Cross an `.await` point.
- Panic or unwind (the borrow flag would remain set, deadlocking future access).

8.6 Why the distinction matters for critical software

1. **Data-race freedom by construction.** Shard-local state is never accessed concurrently. There is no lock, no atomic, no memory-ordering subtlety. If you hold `&mut T`, no one else can access `T`. This is Rust's borrow-checker guarantee extended to the kernel's scheduling model.
2. **Placement is deliberate.** Sending work to a specific LP is an explicit operation (`try_run_on_lp`, `ShardSender::try_send`) and it is not at the mercy of a global work-stealing scheduler. This makes it possible to reason about locality, cache behavior, and latency.
3. **The two container types force a design choice.** When you reach for shared data, you must decide: is this accessed from interrupt context? Use `PerLp<T>` with locking. Is it thread-only? Use `ShardLocal<T>` with zero overhead. There is no "just wrap it in Mutex and move on" ambiguity.
4. **Cross-shard communication is typed and bounded.** A `ShardSender<M>` carries a specific Rust type (not `Box<dyn Any>`, not a byte buffer). The receiver knows what it will get. The channel has fixed capacity; senders handle backpressure. There is no unbounded message queue that can exhaust memory.

9 The Sitas Executor

The sitas executor is the userspace runtime that drives futures on one shard. This chapter explains how it works, how it interacts with the completion queue, and why the unified wait is the central insight of the co-design.

9.1 One executor per shard

Each sitas shard has:

- **One executor** — a cooperative scheduler that polls ready futures within a budget.
- **One ready queue** — tasks whose futures returned `Poll::Pending` and whose wakers have since been invoked.
- **One timer wheel** — futures awaiting deadlines.
- **One CQ partition** — the completion queue on which this shard waits (`CQ_WAIT`).
- **Shard-owned service state** — mutable data that only this executor touches.

The executor runs on one LP-affine thread. When that thread is scheduled onto its LP by the kernel, the executor runs. When the thread blocks (no ready tasks), the LP is free to run another domain's thread.

9.2 The polling loop

The executor's main loop is:

```
loop {
    // 1. Poll ready tasks within a budget.
    while budget_remaining > 0 && ready_tasks.not_empty() {
        let task = ready_tasks.pop();
        match task.future.poll(task.waker) {
            Poll::Ready(output) => { /* task done */ }
            Poll::Pending => { /* will be re-awoken by waker */ }
        }
        budget_remaining -= 1;
    }

    // 2. If tasks remain but budget exhausted, re-queue and yield.
    if ready_tasks.not_empty() {
        ready_tasks.requeue_for_next_tick();
    }

    // 3. No ready tasks? Block in the unified wait.
    if ready_tasks.is_empty() && timer_wheel.is_empty() {
        reactor.wait_for_events(); // -> CQ_WAIT
    }
```

```
}
```

The budget prevents one task (or a tight loop of always-ready tasks) from starving other tasks on the shard. If the budget is exhausted with work remaining, the executor re-queues the remaining tasks and yields (either to other threads on the same LP via the kernel scheduler, or to idle).

9.3 The unified wait — three event sources, one wait point

When the executor has no ready tasks and no pending timers, it calls `CQ_WAIT` on the shard's completion queue. This is the **only** blocking point in the shard's execution.

Three event sources converge on this one wait point:

9.3.1 Kernel completions (CQ entries)

When a submitted operation completes (timer expired, DMA finished, device interrupt processed), the kernel posts a `CompletionEntry` to the shard's CQ ring and wakes the blocked thread. The executor drains the ring:

```
fn drain_cq(&mut self) {
    while let Some(entry) = self.cq_ring.read() {
        let waker = self.waker_registry.take(entry.user_data);
        if let Some(waker) = waker {
            waker.wake(); // task becomes ready
        }
    }
}
```

9.3.2 Endpoint readiness (coalesced wake)

An endpoint can be bound to a CQ. When a message arrives on the endpoint, the kernel posts a coalesced wake to the bound CQ. The executor, on waking, calls the endpoint's receive path to drain the pending messages.

Coalescing means: if 10 messages arrive before the shard wakes, the kernel delivers one wake, not 10. The executor drains all 10 messages in one batch.

9.3.3 Device interrupts (coalesced wake)

A device interrupt object can be bound to a CQ. When the interrupt fires, the kernel posts a coalesced wake. The executor, on waking, calls the device's interrupt handler (or the driver's polling function).

9.4 Task wakeup — from CQ entry to future poll

When the executor drains a CQ entry, it looks up the registered waker for that entry's `user_data` cookie. The cookie was set by the task when it submitted the operation. The waker marks the task ready:

```
// Inside a task's Future::poll:
fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<u64> {
    match self.cap.poll() {
        Ok(Some(completed)) => Poll::Ready(completed.result),
        Ok(None) => {
            // Register waker so we are notified when this completes.
            self.cap.register_waker(cx.waker().clone());
            Poll::Pending
        }
        Err(e) => Poll::Ready(/* error */),
    }
}
```

The waker is a `ShardWaker`: when invoked, it sets the task's ready flag and signals the reactor (if the reactor is currently blocked) to re-enter the polling loop. The reactor then polls the task, which finds its completion ready.

9.5 Cross-shard messaging without the kernel

Within one protection domain, tasks on different shards communicate through sitas typed mailboxes — not through kernel endpoint IPC:

```
// Shard A: send work to Shard B.
let sender: ShardSender<Command> = shard_b.mailbox();
sender.try_send(Command::Process { data: owned_buffer })?;

// Shard B: receive work.
let receiver: ShardReceiver<Command> = shard_b.receiver();
loop {
    let cmd = receiver.recv().await?;
    match cmd {
        Command::Process { data } => { /* handle */ }
    }
}
```

The `ShardSender/ShardReceiver` pair is a bounded, typed, MPSC channel within userspace. Sending does not cross the kernel boundary. The receiving task registers its waker on the channel; when a message arrives, the waker fires and the task re-enters the executor's polling loop.

This is **internal** cross-shard communication (same protection domain, different shards). External communication (different protection domains) uses endpoint IPC (Chapter 5).

9.6 The ShardParker — blocking on native state

When the executor has no ready tasks, it must block its OS thread. The mechanism is the `ShardParker` trait:

```
trait ShardParker {
    fn park(&self);    // Block the current thread.
    fn unpark(&self);  // Wake the blocked thread (from another shard).
}
```

On CharlotteOS, the implementation is:

```
fn park(&self) { syscall::cq_wait(self.cq_id); }
fn unpark(&self) { syscall::cq_wake(self.cq_id, self.target_lp); }
```

The parker maps directly to `CQ_WAIT` / `CQ_WAKE`. There is no intermediate thread pool, no `pthread_cond_wait`, and no busy-loop. The thread parks on native completion and message state.

This is the heart of the "no retrofit tax" claim: the sitas runtime does not emulate async on top of a blocking kernel because the kernel's blocking primitive (`CQ_WAIT`) *already* understands the semantics of "wait for any of several async sources." The executor doesn't need to build a reactor on top of `epoll` or a thread pool; the kernel *is* the reactor.

9.7 Why the sitas executor matters for critical software

1. **Cooperative scheduling means no data races on shard-local state.** Within a shard, only one task runs at a time. The executor polls tasks sequentially. There is no preemption within the shard. This means shard-local mutable state never needs locks.
2. **The single wait point eliminates a class of race conditions.** The executor does not wait on "timer or CQ or endpoint" through three separate blocking calls with three separate check-and-arm sequences. One CQ, one wait, one drain loop. The kernel guarantees the wake is not lost between checking and arming.
3. **No thread-pool overhead.** The executor uses exactly one kernel thread per shard. When that thread blocks, it blocks on the CQ — the same object that will deliver its wake. There is no need for a pool of threads to convert blocking syscalls into futures.
4. **Budgeting prevents starvation.** The polling budget ensures that a shard with many ready tasks makes progress on all of them, not just the one that finishes fastest. This prevents priority-inversion-like behavior within a shard.
5. **Internal messaging avoids unnecessary kernel crossings.** Shards within the same process communicate through typed channels in userspace. Only cross-process IPC crosses the kernel boundary. This keeps the fast path fast and the trusted computing base small.

10 Scheduling and Backpressure

This chapter explains how the kernel schedules threads across LPs, how the block-yield-wake cycle works, and how backpressure propagates through every bounded layer of the system.

10.1 The kernel scheduler

CharlotteOS uses a pluggable scheduler architecture. The system scheduler (`SYSTEM_SCHEDULER`) delegates to per-LP `LpScheduler` instances. The reference implementation is `RoundRobin`:

- A FIFO run queue per LP.
- A per-LP quantum timer (10 ms by default) that drives preemption.
- Operations: `spawn_thread`, `block_thread`, `submit_ready_thread`, `yield_lp`, `abort`.

The scheduler is pluggable: a different scheduling policy (EDF, CFS, fixed priority) can be provided by implementing the `LpScheduler` trait. The rest of the kernel interacts with the scheduler only through the abstract interface.

10.2 The block-yield-wake cycle

This is the fundamental mechanism by which asynchronous operations work. Every async primitive in CharlotteOS — `sleep()`, `wait()`, `CQ_WAIT` — is implemented through this cycle.

10.2.1 Block

A thread blocks by calling `block_thread(tid, &observable)`:

1. The kernel marks the thread `Blocked` in the scheduler.
2. The thread's `Waker` is registered as an observer on the observable event (a completion, a timer, an endpoint, or a CQ).
3. The thread is removed from the LP's run queue.
4. `yield_lp()` saves the thread's register state and stack pointer to its `ThreadContext`.
5. The scheduler selects the next ready thread (if any) and restores its context. The blocked thread is suspended at the point of the yield.

10.2.2 Wake

An event fires (timer expired, completion posted, message arrived):

1. The event's observable fires, calling `Waker::notify(tid)`.
2. `notify` calls `submit_ready_thread(tid)`, which marks the thread `Ready` and adds it to its affinity LP's run queue.

3. If the target LP is idle, a reschedule IPI is sent to wake it. If the target LP is already running, no IPI is needed — the thread will be picked up at the next scheduling point (quantum expiry or voluntary yield).

10.2.3 Resume

At the next scheduling opportunity on the target LP:

1. The scheduler selects the woken thread from the run queue.
2. The thread's saved `ThreadContext` (registers, stack pointer) is restored.
3. Execution resumes immediately after the `yield_lp()` call that originally blocked the thread.
4. From the thread's perspective, `block_thread` returned normally and the event has fired.

10.3 Wake coalescing

Wakes are **idempotent**: calling `submit_ready_thread` on an already-`Ready` or `Running` thread is a no-op. This enables coalescing:

- If 10 messages arrive on an endpoint before the shard wakes, the kernel posts one wake, not 10.
- If 3 timer events fire before the LP processes the timer queue, the kernel delivers them in one batch during the next `process_events`.
- The unified shard wait (Chapter 9) aggregates multiple wake sources onto a single thread. Re-waking that thread is harmless.

The rule is: **enqueue before signaling.** The work is placed in a queue (CQ ring, endpoint queue, timer queue) first; then a wake is delivered. The woken thread finds the work when it drains the queue.

10.4 The quantum timer

Each LP's `RoundRobin` scheduler arms a periodic quantum timer (10 ms default). When it fires:

1. The timer IRQ handler sets a context-switch-pending flag.
2. At the next `yield_lp()` (or at the IRQ handler's tail), the scheduler checks the flag.
3. If set and another thread is runnable, the current thread's context is saved and the next thread's is restored.

An `armed` flag ensures exactly one quantum event is in flight at any time, preventing unbounded timer-queue growth from manual yields.

10.5 The deferred reaper

A thread cannot free its own kernel stack (it is executing on it). When a thread exits:

1. `abort()` moves the dying thread from the thread table to a `DEAD_THREADS` list. The thread is extracted from the table without being dropped.
2. The scheduler performs a context switch to the next thread.
3. **After** the switch (on the new thread's stack), `reap_dead_threads()` drops the dead threads, freeing their stacks and other resources.

This deferred reclamation is the minimal safe discipline for a kernel where threads own their stacks.

10.6 Load rebalancing

By default, threads are pinned to their affinity LP. Migration is opt-in:

- A thread must be marked `migration_safe`.
- Migration is rejected while the thread holds timer events, endpoint waiters, CQ registrations, or other LP-affine ownership constraints.
- Rebalancing uses a **sustained-imbalance window**: per-LP load is sampled over a runtime-adjustable interval rather than reacting to a single instantaneous queue-depth observation.

This conservative approach preserves cache warmth and eliminates migration races (observer-not-found when a completion is posted on a different LP than the one where the waiter was registered). Stable affinity is the correctness default where migration is the performance optimization.

10.7 Backpressure — bounded queues at every layer

Backpressure is a design principle. Every queue in CharlotteOS is bounded, and every sender must handle `WouldBlock`:

Layer	Bounded object	Backpressure signal
Application request	Service endpoint queue	`QueueFull`
Service shard mailbox	`ShardSender<M>`	`Err(m)` returned
Driver endpoint/ring	NIC descriptor ring	Descriptor exhaustion
Kernel submission	Capability table	`WouldBlock`
Hardware	DMA descriptor ring	Ring full

Each layer has an explicit backpressure policy:

```
enum SendPolicy {
    Try,                    // Fail immediately if full.
    Wait,                   // Block the sender until capacity is available.
    Deadline(Instant),      // Block with a timeout.
    DropIfCoalescible,      // Merge with a pending notification (for coalescible
                            //   events like "data available").
}
```

Rules:

- **Terminal completions are never dropped.** The CQ backlog guarantees this (Chapter 7).
- **Coalescible notifications may be merged.** Two "data available" signals before the consumer wakes become one.
- **Control messages normally fail or wait explicitly.** A service registration request that cannot be queued fails with `QueueFull`.
- **Data-plane producers stop before unbounded allocation.** A NIC driver stops accepting TX packets when its descriptor ring is full.
- **Emergency paths require reserved capacity.** The bounded IPI queue reserves slots for mandatory kernel messages (TLB shootdowns, scheduler commands), ensuring they cannot be starved by user work.

10.8 Why scheduling and backpressure matter for critical software

1. **The block-yield-wake cycle is proven correct.** The same mechanism handles sleep, IPC wait, CQ wait, and timer wait. A bug fixed in one path fixes all paths. The cycle is exercised by the async demo and by every self-test that calls `wait()`.
2. **Idempotent wakes eliminate lost-wake races.** Re-waking an already-ready thread is a no-op. This means the kernel can be aggressive about delivering wakes without worrying about double-delivery.
3. **Bounded queues prevent resource exhaustion.** An attacker (or a buggy service) cannot exhaust kernel memory by sending an unbounded number of messages. Every queue has a fixed capacity; every sender must handle `WouldBlock`. The system degrades gracefully — requests are rejected with backpressure, not with an out-of-memory panic.
4. **Stable LP affinity prevents migration races.** Completion delivery is LP-local. A waker registered on LP 2 is signaled on LP 2. There is no window where a completion is posted on LP 1 while the thread is migrating to LP 2 and the observer is not found.
5. **The scheduling policy is pluggable.** The `RoundRobin` scheduler is the default, but a real-time or deadline scheduler can replace it without changing the rest of the kernel. This enables mixed criticality: real-time shards get EDF scheduling; throughput-oriented shards get round-robin.

11 Service Architecture

This chapter explains how protection domains are created, how services are named and discovered, and how the system recovers from failures — all without privileged code knowing what any particular service *does*.

11.1 Protection domains as failure boundaries

A protection domain (Chapter 3) is the unit of both authority and failure. When a service crashes:

- Its capability table is destroyed. Every connection to every client is invalidated atomically.
- Its memory mappings are removed. Outstanding borrows are revoked; the kernel unmaps the pages from the (dead) borrower and restores the lender's access.
- Its threads are aborted. Any thread blocked on a CQ or endpoint is woken with a terminal error.
- Its device grants are recovered. The supervisor regains ownership of any MMIO region and interrupt objects the domain held.

A crash in one domain does not propagate to the kernel or to other domains. The failure is contained by the MMU and the capability system.

11.2 The supervisor — domain lifecycle manager

The **supervisor** is a kernel-side component that creates and destroys protection domains. It does not understand service protocols but it do understand ELF images, memory mappings, and capability grants.

Its operations are:

1. **Load:** parse an ELF image, validate its segments, create a new address space, map `PT_LOAD` segments at their linked virtual addresses, set up a stack with a guard page.
2. **Grant:** populate the domain's config-page with the bootstrap capabilities it needs:
   - A connection to the name service (for service lookup).
   - For driver domains: an `MmioRegion` capability, an `Interrupt` capability.
   - For service domains: an `Endpoint` capability on which to listen.
   - A default CQ handle.
3. **Spawn:** create the initial thread with the domain's entry point and its affinity LP.
4. **Monitor:** register to be notified when the domain exits.
5. **Teardown:** on exit, reclaim all capabilities, revoke mappings, reconcile outstanding operations, and make the address-space ID available for reuse.

Grants are minted kernel-side by the supervisor and never through a syscall. A driver does not ask the kernel for MMIO; the supervisor grants it at creation time based on the device topology it enumerated during boot.

11.3 The name service — discovery without the kernel

The kernel does not know about service names. A userspace **name service** maps human-readable names to connection capabilities:

Registration A service calls `register("driver.nic.v1", endpoint, access_key)` on the name service. The name service stores the endpoint's identity and opens a connection to it.
Generation tracking Each registration bumps a generation counter. If the service restarts and re-registers with the same name, the new generation replaces the old one. Existing connections bound to the old generation fail with `ServiceRestarted`.
Lookup A client calls `lookup("driver.nic.v1", access_key)`. The name service checks the access key (if the service registered one), then delegates a connection capability to the client — typically with `SEND | CALL` rights only.
Access-key gating A registration with a non-zero access key requires lookers to provide the matching key. This is userspace policy: the name service enforces it; the kernel does not participate.

The name service runs as a `no_std` EL0 service process, reached through the bootstrap connection every domain receives at startup.

11.4 Service restart and recovery

When a service crashes and the supervisor restarts it:

1. **Device reset.** If the service was a driver, the supervisor resets the device before re-granting it to a new driver instance. The new driver sees a clean hardware state.
2. **Generation increment.** The name service bumps the service's generation. All old connections are invalidated.
3. **Stale connection failures.** Clients holding connections to the old instance receive `ServiceRestarted` on their next call attempt. They re-lookup the service and receive a connection to the new instance.
4. **Outstanding operation reconciliation.** The supervisor reconciles any operations the old domain had submitted but not completed: cancellations are posted, borrowed memory is returned, DMA buffers are recovered.
5. **Fresh bootstrap.** The new domain receives a fresh config-page with freshly minted capabilities. It does not inherit the previous instance's capability table, reply tokens, or DMA ownership.

A restarted service must not silently inherit outstanding authority or in-flight operations from the previous instance. The kernel and the supervisor guarantee this.

11.5 The overall system topology

At steady state, a CharlotteOS system consists of:

- **The kernel** — capabilities, endpoints, memory objects, CQs, scheduling, interrupt routing.
- **The supervisor** (kernel-side) — domain lifecycle, ELF loading, device enumeration.
- **The name service** (userspace) — service registration, lookup, access-key gating, generation tracking.

- **Service processes** — one or more per logical function: NIC driver, TCP/IP stack, filesystem, Raft consensus, application servers.
- **Client processes** — application code that looks up services and invokes them through capabilities.

Every component beyond the kernel, supervisor, and name service is a replaceable userspace process. A different NIC driver, a different filesystem, a different network stack — these are configuration choices, not kernel modifications.

11.6 Why service architecture matters for critical software

1. **Failure containment is architectural.** If the NIC driver crashes, the TCP/IP service loses connectivity but does not crash. If the filesystem crashes, applications holding file handles get errors. If the name service crashes, lookup fails but existing connections remain valid. No single service crash takes down the system.
2. **Recovery is defined, not ad-hoc.** The restart sequence (device reset, generation bump, connection invalidation, operation reconciliation) is the same for every driver. A new driver implemented to the same endpoint protocol replaces the old one without any special-case recovery code.
3. **Policy lives in userspace.** The kernel enforces mechanism (capabilities, endpoints, memory objects). Userspace enforces policy (who can register what name, who can look up what service, how many restarts are allowed, what log level to use). The kernel stays small; policy is auditable and replaceable.
4. **The bootstrap contract is explicit and minimal.** Every domain starts with exactly the capabilities it needs and no more. The config-page contract defines what a domain receives; there is no ambient authority to discover or exploit.

12 Userspace Drivers

In CharlotteOS, device drivers are userspace processes. They receive only the resources they need, they communicate through endpoints, and their failures are contained. This chapter explains the userspace driver model.

12.1 The problem with kernel drivers

In a monolithic kernel, a driver has access to everything: all physical memory, all interrupt vectors, all kernel data structures. A bug in a driver — a buffer overflow, a use-after-free, a race condition — can corrupt the kernel and crash the system. This is not merely a theoretical concern: driver bugs are the dominant cause of operating system crashes in most operating systems.

The microkernel solution is to move drivers to userspace. But simply moving them is not enough: a userspace driver that can still map arbitrary physical memory or receive arbitrary interrupts is still dangerous. The driver must be **granted** only what it needs, and the grants must be mediated by the kernel.

12.2 The DriverGrant

When a driver domain is created, the supervisor delivers a `DriverGrant` through the config-page contract:

```
struct DriverGrant {
    device: DeviceCap,        // identity and lifecycle of the device
    mmio: Vec<MmioRegionCap>, // register windows, page-granular
    interrupts: Vec<InterruptCap>, // IRQ sources for this device
    dma_domain: Option<DmaDomainCap>, // DMA translation, if IOMMU present
    bootstrap_endpoint: EndpointCap,  // endpoint on which to serve clients
}
```

The driver receives **exactly** the MMIO register windows and interrupt vectors it needs. It cannot:

- Map arbitrary physical memory.
- Receive interrupts not assigned to its device.
- Access other devices' MMIO regions.
- Perform DMA without explicit pinning into its DMA domain.

12.3 MMIO delegation

An `MmioRegion` capability represents a page-granular device register window. The driver maps it into its address space as Device-nGnRnE memory (user-accessible, execute-never, strongly ordered on AArch64; uncacheable on x86-64).

The mapping is a kernel-mediated operation: the driver calls `mmio_map(region_cap, virtual_address)`. The kernel validates the capability, creates the page-table entries with the correct memory attributes, and returns. The driver then performs **direct EL0 MMIO** — load and store instructions to the mapped virtual addresses, with no kernel involvement on the fast path.

This is the key performance property: the common case (reading a device register, writing to a FIFO) is one CPU instruction. There is no syscall overhead per MMIO access.

12.4 Interrupt delivery

An `Interrupt` capability represents a routed interrupt source. The driver binds it to its completion queue:

```
interrupt_bind(int_cap, cq_id);
```

When the interrupt fires:

1. The IRQ dispatcher reads the per-interrupt route table atomically.
2. It masks the source at the interrupt controller (GICv3 on AArch64, x2APIC on x86-64).
3. It atomically bumps a coalescing counter.
4. It pushes a wake onto a deferred queue. The actual `completion::wake` (which takes locks) is performed from thread context by `yield_lp` and the LP idle loop.
5. The driver's shard wakes.
6. The driver reads the device's status register (direct EL0 MMIO) to determine the interrupt cause.
7. The driver processes the interrupt and unmasks the source.

This is a lock-free fast path: the interrupt context does very little work (atomic counter bump, deferred-wake enqueue), deferring the rest to thread context.

12.5 The reference UART driver

The PL011 UART driver demonstrates the full lifecycle:

- **Direct MMIO writes to the transmit FIFO.** No syscalls.
- **Interrupt-driven deferred reads.** The driver retains a reply token for a pending read. When the RX interrupt fires, the driver reads the receive register and completes the deferred reply.
- **Cooperative shutdown.** On graceful exit, the driver unmap its MMIO region and closes its interrupt.
- **Crash recovery.** If the driver terminates without releasing its device caps (panic, infinite loop, killed by supervisor), the teardown path:
  1. Reclaims the MMIO region (unmaps it, returns to the kernel).
  2. Unroutes the interrupt.
  3. Reconciles outstanding operations: any pending reply tokens are invalidated; any borrowed memory is returned to lenders.
  4. Resets the device hardware.

  A restarted instance with fresh grants resumes under a bumped generation.

12.6 DMA and the IOMMU

For DMA-capable devices (network cards, storage controllers), a userspace driver needs more: the ability to tell the device "read from physical address X" or "write to physical address Y." Without an IOMMU, this is a safety gap: a malicious or buggy driver could program the device to DMA into kernel memory or another domain's pages.

The architecture anticipates two levels of isolation:

- **Without IOMMU:** the driver is trusted with DMA. It still receives only the MMIO and interrupt it needs, and it still runs in a separate address space, so a CPU-side bug cannot read kernel memory. But a DMA programming bug can violate memory isolation.
- **With IOMMU/SMMU:** the DMA domain capability binds the device to a specific IOMMU translation table. The driver can only DMA into pages explicitly pinned into its DMA domain. The kernel validates every pin/unpin through the capability. The IOMMU hardware enforces the rest — even a malicious driver cannot DMA outside its domain.

The driver API is the same in both cases: `dma_pin(memory_object)` and `dma_unpin(memory_object)`. Without an IOMMU, these are no-ops (the kernel trusts the driver). With an IOMMU, they manipulate the translation table.

12.7 Driver service layering

Complex I/O stacks decompose into separate failure and authority domains:

```
Application
    -> socket endpoint
Network service
    -> packet endpoint
NIC driver
    -> MMIO / DMA / IRQ
Hardware
```

The NIC driver knows about hardware registers and descriptor rings, the network service knows about protocols and sockets, and the application knows about its business logic. A crash in any layer is contained: the NIC driver crashes, the network service sees `ServiceRestarted`, and reconnects to a fresh instance.

12.8 Why userspace drivers matter for critical software

1. **Driver bugs are contained.** A buffer overflow in a NIC driver corrupts that driver's address space, not the kernel's. The driver crashes (and restarts); the system stays up.
2. **The privilege model is grant-by-grant.** A driver receives exactly the resources it needs. There is no "driver has access to everything because it's in the kernel" default.
3. **Direct MMIO keeps the fast path fast.** Reading a device register is one load instruction. No syscall, no kernel transition, no context switch. The driver is in userspace, but the performance cost is near zero.

4. **Restart is automated and defined.** The supervisor handles driver restart (device reset, grant re-issuance, generation bump). Driver authors do not write recovery code.
5. **Hardware-isolated DMA is planned.** The IOMMU integration path is designed into the architecture from the start. When IOMMU support is enabled, DMA-capable drivers gain hardware-enforced memory isolation without API changes.

13 Networking

CharlotteOS deliberately departs from the traditional UNIX networking model. Rather than treating TCP/IP as the foundation of all communication, CharlotteOS treats networking as **an extension of the operating system's message passing and capability model.**

The guiding principle is:

> **Remote IPC is local IPC with a different transport.**

13.1 Why not sockets?

The Berkeley socket API (`socket()`, `connect()`, `send()`, `recv()`) has served the Internet for decades. But it is not the right foundation for a capability-based microkernel:

- **Sockets identify hosts, not services.** `10.2.3.17:443` tells you where to connect, not what you are talking to. The mapping from address to service is out-of-band (DNS).
- **Sockets are byte streams, not messages.** Message boundaries are lost. Every protocol invents its own framing on top of TCP's undifferentiated stream.
- **Sockets carry ambient authority.** Any process can open a socket to any address. There is no capability check; authority comes from being able to reach the network, not from being granted a connection.
- **Sockets couple applications to transports.** Code written against TCP cannot transparently use shared memory for local communication or RDMA for high-performance communication.

CharlotteOS replaces all of this with a layered model where each layer has a single responsibility.

13.2 The native protocol stack

```
Applications
        |
Capability Invocation
        |
Distributed Objects
        |
RPC
        |
Reliable Message Layer
        |
Ethernet Frame Transport
        |
VirtIO / NIC Driver
```

13.2.1 Ethernet — generic frame transport

The NIC driver exports raw Ethernet frames. It knows about MAC addresses, MTU, and frame CRCs. It knows **nothing** about IP, TCP, RPC, or services. Its endpoint protocol has two operations:

- `OP_SEND(frame_buffer)` — transmit a frame via a moved memory object.
- `OP_RECV` — deferred receive: the caller retains a reply token; the driver replies with a received frame buffer.

Ethernet is a generic frame transport that can carry IP, ARP, LLDP, custom protocols, or CharlotteOS native messages. The driver provides the frames and higher layers interpret them.

13.2.2 Reliable message layer

This layer provides sequenced, acknowledged, reliable delivery of **messages** (not streams) over the raw frame transport. It handles:

- Sequencing (each message has a monotonic sequence number).
- Acknowledgements (the receiver confirms delivery).
- Retransmission (unacknowledged messages are resent).
- Fragmentation (messages larger than MTU are split and reassembled).
- Batching (multiple messages in one frame, one ACK for multiple messages).
- Congestion control (bounded in-flight windows).

The abstraction exported is a **reliable message**, not a byte stream. This is the building block for RPC, consensus protocols, and distributed objects.

13.2.3 RPC layer

The RPC layer provides request/reply semantics over the reliable message layer:

- **Synchronous-looking futures:** `service.call(opcode, args).await?`
- **Asynchronous invocation:** the `Future` is backed by a pending RPC call; the shard continues other work while waiting.
- **Object references:** capabilities can be serialized and sent over RPC (delegation across machines).
- **Serialization:** typed protocol crates handle encoding/decoding between Rust types and the wire format.

13.2.4 Distributed objects

At the top of the stack, applications interact with **capabilities** — the same abstraction they use for local IPC:

```
let payroll: Capability<PayrollService> = lookup("Payroll")?;
payroll.call("calculate_salary", employee).await?;
```

Whether the Payroll service runs on the same machine (local endpoint IPC) or on a different machine (remote RPC over reliable messages over Ethernet) is **transparent to the application**. The capability lookup returns a connection; the transport layer behind that connection is an implementation detail.

13.3 TCP/IP as a compatibility service

TCP/IP remains fully supported — as a userspace compatibility service, not the native programming model. A smoltcp adapter implements the `phy::Device` trait over the NIC driver endpoint:

```
// smoltcp adapter:
fn receive(&mut self, now: Instant) -> Option<(IpPacket, ...)> {
    // Calls OP_RECV on the NIC driver endpoint.
}

fn transmit(&mut self, now: Instant) -> Option<...> {
    // Calls OP_SEND with a memory object.
}
```

The TCP/IP stack itself — a standard smoltcp 0.13 instance — needs no modification. It is wrapped in a userspace service that registers as `"tcpip"` and serves clients through endpoint IPC. Legacy software that requires TCP/IP connects to this service.

13.4 Transport independence

The reliable message layer is transport-agnostic. It can run over:

- **Ethernet** — for inter-machine communication.
- **Shared memory** — for co-located processes on the same machine, where the kernel can set up shared memory regions instead of going through the NIC.
- **Loopback** — for same-machine same-stack communication.
- **RDMA** — for high-performance cluster interconnects.
- **PCIe NTB** — for backplane communication in multi-socket systems.

The RPC layer and everything above it are unchanged regardless of the transport. This is the same principle that makes local IPC transparent to the application when it moves from same-process to same-machine to different-machine.

13.5 Zero-copy networking

The network stack avoids copying whenever possible:

- **Receive path:** the NIC driver receives a buffer from hardware. That buffer (a memory object) is passed up the stack by transferring ownership — first to the message layer (for reassembly), then to the RPC layer (for deserialization), then to the application. No intermediate copies.

- **Transmit path:** the application creates a buffer. It is passed down by moving ownership. The NIC driver DMA's it directly from the application's pages.
- **Buffer recycling:** after the application consumes a received buffer, it returns the (now empty) memory object to the driver's RX pool. The driver re-pins it for DMA and the cycle repeats.

This is the memory-object transfer model (Chapter 6) applied to networking.

13.6 Why the networking architecture matters for critical software

1. **Remote IPC = local IPC.** An application that invokes a capability does not know or care whether the service is local or remote. This means:
   - Services can be relocated without application changes.
   - Testing can substitute a local mock for a remote service.
   - The network boundary is not a source of bugs that only appear in production.
2. **Capability-based, not address-based.** An application invokes `PayrollService`, not `10.2.3.17:443`. Authority is a capability, not reachability. A compromised application that can reach the network cannot fabricate a connection to a service it was not granted.
3. **Zero-copy without loss of safety.** Pages move by changing ownership, not by copying. The MMU (and IOMMU) enforce that the sender cannot access moved data. This is faster than a socket and safer than shared memory.
4. **Message boundaries are preserved.** The native abstraction is a message with identity, ownership, metadata, and boundaries. There is no need for framing protocols on top of TCP streams. This simplifies protocol implementations and eliminates framing bugs.
5. **TCP/IP is available but not mandatory.** Legacy software connects to the TCP/IP service. Native software uses the capability-based stack. The transition is incremental: a service can expose both a native endpoint and a TCP/IP socket through the compatibility service.

14 Consensus and Replication

Distributed consensus is often implemented as an application-level library talking to sockets. CharlotteOS treats it as a **generic capability service**: a replicated state machine exported through the same endpoint IPC that local services use. This chapter explains the architecture and why it is a natural fit.

14.1 Raft as a capability service

The Raft consensus protocol provides a replicated, fault-tolerant state machine across a cluster of nodes. In CharlotteOS, each Raft node is an EL0 service process that:

- **Registers with the name service** as `"raft-<id>"`.
- **Obtains connection capabilities to its peers** through the name service (or through seed configuration at bootstrap).
- **Communicates with peers through endpoint IPC.** VoteRequest and AppendEntries travel as scalar messages; log payloads travel as memory objects (Move transfer for zero-copy replication).
- **Uses completion queues for timers.** Election timeouts and heartbeat intervals use `submit_timer()` — a first-class completion primitive — not polling loops or `sleep()`.
- **Serves client commands through its endpoint.** A client submits a command via `OP_CLIENT_COMMAND`; the leader replicates it, commits it, applies it to the state machine, and replies.

The Raft core (`catten-graft`) is a generic, transport-agnostic library. It implements the `RaftTransport` trait, which abstracts peer communication behind a few methods:

```
trait RaftTransport {
    async fn send_vote_request(&self, peer: PeerId, req: VoteRequest)
        -> Result<VoteResponse, TransportError>;
    async fn send_append_entries(&self, peer: PeerId, req: AppendEntries)
        -> Result<AppendResponse, TransportError>;
    // ...
}
```

The CharlotteOS implementation of this trait uses endpoint IPC. A future implementation could use the reliable message layer for cross-machine communication — the consensus core would not change.

14.2 Why this is better than socket-based Raft

14.2.1 Authority, not addresses

Once the cluster is running, peers are identified by name service entries, not `IP:port`. A node that has joined the cluster receives connection caps for its peers through the name service:

```
let peer_1: ConnectionCap = name_service.lookup("raft-node-1")?;
let peer_2: ConnectionCap = name_service.lookup("raft-node-2")?;
```

No IP configuration. No DNS. The authority to communicate with a peer is the capability; without it, you cannot even address the peer.

14.2.2 Transport transparency

The transport is behind the `RaftTransport` trait. A connection capability routes through local IPC (same machine), Ethernet frames (same network), or a future RDMA transport (high-performance cluster) — the consensus core never changes. This means:

- Development and testing can run a multi-node Raft cluster on one machine with local IPC.
- Deployment can switch to cross-machine networking by changing how peers are resolved, not by changing the Raft code.
- The cluster can be heterogeneous: some peers communicate via shared memory, others via Ethernet.

14.2.3 Zero-copy log replication

Locally, log payloads can transfer via Move memory objects: the kernel flips page-table ownership instead of copying bytes. A follower that receives a Move attachment gets the exact same physical pages the leader held — no intermediate buffer, no copy.

When going over the network, the wire format supports this; the transport implementation decides whether to use scalar IPC (small payloads) or memory-object RPC (large payloads) at the ends.

14.2.4 Capability-based security

A client invokes the Raft service only if it possesses a connection cap. The service manager attenuates rights: a typical client receives only `CALL` for `OP_CLIENT_COMMAND`, not for `OP_ADD_SERVER` or `OP_REMOVE_SERVER`. There is no ambient authority, no IP-based ACL, and no "any process on the cluster network can propose commands."

14.2.5 Failure isolation

Each Raft node runs in a separate protection domain. A crash in one node does not affect the others. The crashed node restarts (via the supervisor), receives fresh grants, recovers its durable state, and rejoins the cluster. Stale client connections to the old instance fail with `ServiceRestarted`; clients re-lookup and receive connections to the new instance.

14.3 Cluster bootstrap — the seed problem

Like any consensus system, a new node must contact at least one existing cluster member. The difference is that the seed is expressed in terms the OS already understands — service names and capabilities — rather than IP addresses.

14.3.1 Static seeds (current, simplest)

Launch-manifest records carry the node and seed-peer service names:

```
node-id = "r4"
peer-id = "r1"
peer-id = "r2"
peer-id = "r3"
```

The node calls `OP_LOOKUP("raft-r1")` to obtain a connection cap. If the seed is on the same machine, the name service returns the cap directly. If remote, the distributed name service resolves it.

14.3.2 Pre-shared cluster capability (small kernel change)

A cluster administrator provisions a "cluster capability" — a connection to a known cluster member — and delivers it to new nodes via the config page. No name lookup required; the node already holds a first-class connection.

14.3.3 Ethernet broadcast discovery (zero-configuration)

On a single Ethernet segment, a joining node broadcasts a frame with a well-known EtherType carrying a "Raft peer discovery" payload. Existing members respond with their service names. The joining node looks them up through the name service to obtain attested connection caps. This works without any seed configuration.

14.3.4 Distributed name service (end state)

Once the cluster is running and the name service is itself replicated through Raft, a new node needs only a single seed to discover all peers through the consistent name registry. The name service is a `StateMachine` implementation on the Raft substrate.

14.4 The generic StateMachine trait

Raft replicates an opaque command log. The meaning of those commands is provided by a pluggable `StateMachine` trait:

```
trait StateMachine {
    fn apply(&mut self, command: Vec<u8>) -> Result<Vec<u8>, ApplyError>;
    fn snapshot(&self) -> Vec<u8>;
    fn restore(&mut self, snapshot: Vec<u8>);
}
```

Different services are different state machines on the same Raft substrate:

- **Distributed name service:** commands are "register(name, endpoint)" and "unregister(name)". The state is a map from names to endpoint identities and generations. Once replicated, service discovery is available cluster-wide.
- **Distributed lock service:** commands are "acquire(lock_id)" and "release(lock_id)". The state is a set of held locks.
- **Cluster configuration store:** commands are arbitrary key-value writes. The state is a key-value map.

None of these state machines know about sockets, IP addresses, or TCP. They implement `apply()` and `snapshot()`. The Raft substrate handles replication, leader election, and fault tolerance.

14.5 Relationship to the name service

The existing single-node name service becomes a replicated service when it runs as a `StateMachine` on top of Raft:

- `OP_REGISTER` and `OP_LOOKUP` are submitted as commands through the Raft leader, replicated to the log, and applied at each node.
- Service generations are replicated consistently. A restart on node 3 cannot create a split-brain where node 1 sees generation 5 and node 2 sees generation 3.
- Access keys are enforced consistently. Revocation propagates through the replicated log.

14.6 Election timers as first-class completions

In a traditional Raft implementation, election timeouts are implemented with `sleep()` or a polling loop:

```
// Traditional: busy wait or sleep
while !timeout_elapsed() {
    sleep_ms(10); // or just spin
}
```

In CharlotteOS, they use the completion system:

```
// CharlotteOS:
let timer_id = submit_timer(timeout_duration)?;
let result = wait_on_cq(timer_id).await?;
// Timer fired; start election.
```

The timer is a kernel-managed operation. The shard parks on its CQ. When the timer fires, the executor wakes, drains the CQ entry, and resumes the election task. No polling loop, no thread dedicated to sleeping.

14.7 Why consensus as a service matters for critical software

1. **Consistency primitives are OS-level, not library-level.** A service that needs replication does not integrate a Raft library and manage its own sockets, timers, and persistent storage. It implements `StateMachine` and the OS handles the rest.

2. **Capabilities, not IP addresses, identify peers.** The cluster configuration is a set of capabilities, not a list of `host:port` pairs. A network reconfiguration (new NIC, new subnet, move to RDMA) does not require a cluster reconfiguration.
3. **Timers are first-class and reliable.** An election timeout that is lost because the polling loop missed its window is a correctness bug (it can cause spurious leader changes or extended unavailability). Kernel-managed timers delivered through the non-lossy CQ guarantee delivery.
4. **Failure isolation extends to the consensus layer.** A Raft node is a userspace process. It can crash, restart, and recover without affecting the kernel or other Raft nodes. The supervisor handles the lifecycle; the Raft protocol handles the consensus.
5. **The distributed name service is a natural extension.** Once Raft is available as a service, all other services that need cluster-wide consistency (name service, lock service, configuration) are StateMachine implementations on the same substrate. There is one consensus implementation to harden and audit.

15 Persistent Storage

CharlotteOS treats storage as a layered stack of userspace services, each speaking a well-defined protocol through endpoint IPC. No part of the storage stack runs in the kernel — the kernel grants an NVMe driver an MMIO window and an interrupt, and the rest is composed in userspace.

15.1 The storage stack

```
Filesystem ("fs")           ← charlotte-protocol-fs
    |
Object store ("obj")        ← charlotte-protocol-objstore
    |
Block driver ("blk0")       ← charlotte-protocol-block
    |
NVMe controller             ← MMIO + DMA + MSI-X (granted at boot)
```

15.2 NVMe block driver

The NVMe driver is a userspace EL0 service implementing the NVM Express 1.4 base specification. It receives an `MmioRegion` capability covering the controller's BAR0 register window and an `Interrupt` capability for I/O completion signalling. It never names a physical address or interrupt vector — both arrive through the config-page contract.

15.2.1 Initialisation

The driver follows the standard NVMe initialisation sequence:

1. **Controller reset:** disable CC.EN, wait for CSTS.RDY = 0.
2. **Admin queue setup:** allocate physically contiguous memory for the Admin Submission Queue (ASQ, 32 entries) and Admin Completion Queue (ACQ, 32 entries). Write AQA, ASQ, and ACQ registers. Enable the controller and wait for CSTS.RDY = 1.
3. **Identify controller:** submit an Identify command (CNS=1) through the admin SQ. Read capabilities, version, and supported features.
4. **Set Features (Number of Queues):** declare how many I/O queue pairs the driver intends to use. QEMU's NVMe controller requires this before accepting I/O queue creation commands.
5. **Identify namespace:** submit an Identify command (CNS=0, NSID=1) to read the namespace size (NSZE) and the active LBA format (FLBAS), which determines the block size.
6. **Create I/O Completion Queue:** allocate memory, submit a Create I/O CQ admin command. The CDW11 register must set bit 0 (PC = Physically Contiguous) to 1 — this is required by the NVMe specification and enforced by QEMU's device model.
7. **Create I/O Submission Queue:** allocate memory, submit a Create I/O SQ admin command, associating it with the I/O CQ created in the previous step.

After initialisation the driver registers `"blk0"` with the name service and enters its main loop, blocking on `cq_wait` for endpoint messages and I/O completions.

15.2.2 I/O path

OP_READ receives a BorrowWrite memory attachment from the caller. The driver reads the physical address of the attached pages via `memory_get_phys`, constructs an NVM Read command with the starting LBA, block count, and PRP1 pointing to the caller's pages, and submits it to the I/O SQ. The NVMe controller DMAs the data directly into the caller's pages. When the completion arrives (polled via the I/O CQ), the driver replies and the kernel revokes the borrow, returning the now-filled buffer to the caller.

OP_WRITE receives a Move memory attachment. The driver reads the physical address, constructs an NVM Write command, and submits it. The controller DMAs from the caller's pages. On completion the driver replies; the buffer was moved and is consumed.

OP_FLUSH submits an NVM Flush command to ensure all previously written data reaches durable media before replying.

15.2.3 Lessons from bringup

The NVMe driver surfaced several non-obvious requirements:

- **CDW11 bit 0 (PC) must be 1.** QEMU's NVMe device model requires physically contiguous queues. Without this bit, the Create I/O CQ command returns status 0x4002 regardless of all other parameters.
- **SQE layout is exacting.** The Command Identifier occupies bits 31:16 of DW0, the opcode occupies bits 7:0. The NSID shares DW0's high bits with DW0 in the first u64. PRP1 is at byte offset 24, CDW10-11 at byte offset 40. Getting any of these wrong produces silent failures.
- **Doorbell stride must be derived from CAP.DSTRD.** On QEMU this is 0 (stride = 4 bytes), but the architecture supports strides up to $4 \ll 15$ bytes.
- **MMIO mapping must cover the doorbell region.** The NVMe controller registers occupy offsets 0x0000–0x003F within BAR0, but doorbell registers start at offset 0x1000. A single-page MMIO grant is insufficient — at least two pages are needed.
- **On TCG, `cq_wait` is required for VM exits.** A pure spin-loop prevents QEMU from processing I/O because the guest never yields. The driver must block on `cq_wait` to let the host process NVMe commands.

15.3 Block device protocol

The block protocol (`charlotte-protocol-block`) is a minimal `no_std` crate defining the endpoint IPC interface between block device consumers and drivers:

Opcode	Name	Description
1	OP_INFO	Query block size and total block count. Scalar call/reply.
2	OP_READ	Read blocks into a BorrowWrite memory object.

3	OP_WRITE	Write blocks from a Move memory object.
4	OP_FLUSH	Flush write cache to durable media.
5	OP_TRIM	Discard a block range (optional).

The protocol is transport-agnostic: the same interface works for NVMe, AHCI, virtio-blk, or a ramdisk. Consumers speak `OP_READ`/`OP_WRITE` without knowing the underlying hardware.

15.4 Object store

The persistent object store (`charlotte-protocol-objstore`) provides crash-safe, dynamically-sized blob storage on top of a block device. Objects are identified by 64-bit monotonically-increasing IDs.

15.4.1 On-disk layout

```
LBA 0:     superblock (magic, version, generation, next_object_id)
LBA 1:     free block bitmap (1 bit per block)
LBA 2--33: object directory (512 entries × 32 bytes:
             id | flags | size_bytes | first_extent_lba)
LBA 34+:   data extents (linked list of blocks per object)
```

15.4.2 Crash safety

New data extents are written to disk first, then the directory entry's `first_extent_lba` pointer is updated atomically (a single-block write). If a crash occurs between writing the extent and updating the directory, the orphaned blocks are unreachable but the object reverts consistently to its previous state. No journal, no double-writes — the pointer swap is the commit.

15.4.3 Operations

- **CREATE** — allocate a directory slot, return a new object ID.
- **WRITE** — replace the object's content. The caller moves a memory object; the store writes it to a new extent and updates the directory pointer.
- **READ** — return the object's data as a moved memory object.
- **DELETE** — mark the directory entry as free, return the extent blocks to the free bitmap.
- **FLUSH** — flush the underlying block device.

15.5 Native filesystem

The filesystem (`charlotte-protocol-fs`) builds hierarchical naming on top of the object store. Directories and files are stored as objects; the root directory is a fixed object ID (100).

15.5.1 Directory encoding

Each directory object stores entries as a packed sequence:

```
[name_len: u32 LE][name: UTF-8][file_id: u64 LE][flags: u32 LE][size: u64 LE]
```

A `name_len = 0` marker terminates the list. Flags distinguish directories (bit 0 set) from files (bit 0 clear).

15.5.2 Path resolution

Paths are walked client-side by chaining `OP_LOOKUP` calls. The filesystem service does not understand path strings — it only resolves single-component lookups within a given directory. This keeps the filesystem protocol minimal: six operations (LOOKUP, CREATE, READ, WRITE, DELETE, LIST) plus FLUSH.

15.5.3 Host inspection

A Python tool (`scripts/fs-inspect.py`) reads the raw disk image and walks the object store and filesystem on-disk structures. It provides tree view, hex dump, cat, and format commands, enabling post-mortem debugging without booting CharlotteOS:

```
python3 scripts/fs-inspect.py os-images/nvme-disk.img tree
python3 scripts/fs-inspect.py os-images/nvme-disk.img cat /raft/state
```

15.6 Why this matters for critical software

1. **Zero kernel code in the storage path.** A bug in the NVMe driver, the object store, or the filesystem corrupts that service's address space, not the kernel. The kernel grants MMIO and interrupts; everything else is userspace.
2. **Transport independence.** The block protocol abstracts the hardware. The same object store and filesystem work over NVMe, AHCI, virtio-blk, or a network block device — the kernel does not need to know which.
3. **Crash-safe by construction.** The object store's atomic pointer-swap commit eliminates the need for journaling at the storage layer. A crash during a write leaves the previous version intact.
4. **Capability-based access.** A client that holds a connection to `"blk0"` can read and write blocks. A client that does not hold that connection cannot. There is no ambient filesystem namespace to escape.
5. **Host-inspectable.** The disk image is a raw file that can be examined with standard tools and the included Python inspector. No proprietary on-disk format, no opaque blobs.

16 Live Service Upgrade

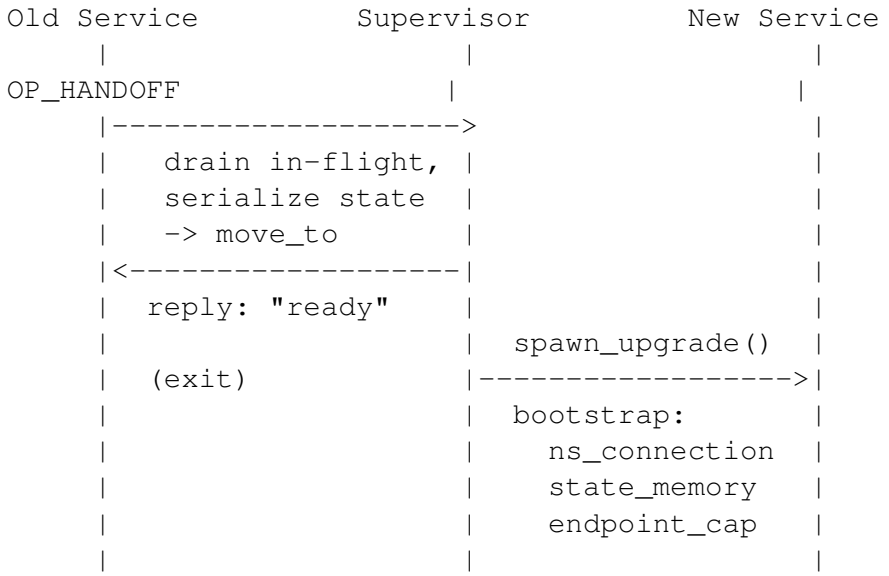
CharlotteOS can replace a running service without losing its state. This is **restart-with-state**, not true zero-downtime: in-flight requests to the old instance fail during the handoff window and clients must retry. But the new instance resumes with the old instance's state intact, and clients reconnect transparently through the name service's generation tracking.

16.1 The four primitives

Live upgrade builds on four primitives already implemented in the kernel and name service:

1. **Memory-object ownership transfer.** The kernel's `memory::object::move_to` moves a page-backed memory object from one address space to another. The sender loses access; the receiver gains it. This is the vehicle for service state.
2. **Generation tracking.** The userspace name service records an instance generation that increments on restart. A client calling through a stale connection from the previous generation receives `EndpointClosed`. Re-looking up the name returns the new generation and a fresh connection to the new instance.
3. **EndpointClose and Cancelled.** When a service domain exits, the kernel closes its endpoint. Queued requests that were never delivered complete as `EndpointClosed`. Delivered calls whose server exited before replying complete as `Cancelled`. Clients see a deterministic failure, not a hung request.
4. **Bootstrap capability delivery.** The supervisor writes initial capabilities to the config page before a domain starts. For a live upgrade, the bootstrap contract is extended: in addition to the name service connection, the new domain receives the old service's state (as memory objects) and the old service's endpoint capability.

16.2 The handoff protocol



```
          |                    |         new service reads state,
          |                    |         takes over endpoint,
          |                    |         registers (new generation)
          |                    |                    |
                    clients see EndpointClosed
                    -> re-lookup -> fresh connection
                    -> resume from where they left off
```

The old service receives an `OP_HANDOFF` request (typically from the supervisor or a service manager). It drains any in-flight work, serialises its mutable state into memory objects, moves those objects to the supervisor's address space, and replies "ready" before exiting.

The supervisor then spawns the new service instance with an extended bootstrap contract: the name service connection, the handoff state memory objects, and the old service's endpoint capability. The new service deserialises the state, takes over the endpoint, and re-registers under the same name. The name service bumps the generation, invalidating all stale client connections.

16.3 What the supervisor provides

```
pub struct UpgradeGrant {
    pub state_caps: Vec<MemoryObjectCap>,
    pub endpoint_cap: CapabilityId,
}

pub fn spawn_upgrade(
    image: &[u8],
    name_service: &NameServiceHandle,
    old_domain: ServiceDomain,
    grant: UpgradeGrant,
) -> ServiceDomain;
```

`spawn_upgrade` loads the new service ELF, moves the state memory objects to the new domain, delegates the old endpoint cap to the new domain, writes the extended bootstrap config page, starts the new domain, and tears down the old domain (reclaiming all remaining resources).

16.4 What the service must implement

The old service must handle `OP_HANDOFF`:

```
OP_HANDOFF => {
    // Stop accepting new work.
    // Serialize all mutable state into memory objects.
    // Move those objects to the supervisor's address space
    //   (the supervisor passes its ASID in the handoff call).
    // Reply "ready", then exit.
}
```

The new service reads the extended bootstrap contract:

```
fn main(ctx: Context) -> ! {
    let ns_connection = ctx.bootstrap_cap();
    let state_count = ctx.handoff_count();
    let state = ctx.handoff_state_cap();
    let endpoint = ctx.handoff_endpoint_cap();

    // Deserialize state, register the endpoint under the
    // same name (NS bumps generation), start serving.
}
```

16.5 What clients see

A client holding a stale connection from the previous generation sees a clean failure and retries:

```
// Before upgrade:
let conn = ns.lookup("payroll")?;       // generation 3
payroll.call(OP_CALCULATE, employee_id); // in-flight

// During upgrade: call completes as EndpointClosed.
// Client retries:
let conn = ns.lookup("payroll")?;       // generation 4
payroll.call(OP_CALCULATE, employee_id); // reaches new instance
```

The new instance has the old instance's state, so it can resume processing where the old instance left off. From the client's perspective, a single request failed and was retried — the rest of the session continues uninterrupted.

16.6 Restart-with-state vs. zero-downtime

This design is **restart-with-state**: in-flight requests to the old instance fail during the handoff window, and clients must retry. This is sufficient for the vast majority of services — the handoff window is measured in milliseconds, and clients already handle transient failures through the name service's generation tracking.

True zero-downtime would require one of:

- **Queue migration:** the old service's endpoint queue would need to be transferred to the new service atomically, so no request is ever rejected. This is a future kernel feature.
- **Sidecar model:** the new service starts alongside the old one, both sharing the same endpoint, and the old service stops accepting new work. The kernel does not currently support multiple owners for one endpoint.

16.7 Relationship to crash recovery

The live upgrade path is the graceful variant of the crash-recovery path already exercised by the UART driver test (Chapter 12). In both cases, the supervisor reclaims device authority, invalidates stale connections, and spawns a new instance with a bumped generation. The difference is that live upgrade transfers state explicitly through memory objects before the old instance exits, rather than starting the new instance from scratch.

16.8 Why this matters for critical software

1. **Services can be upgraded without losing state.** A bug fix or feature update to a filesystem, network stack, or consensus node does not require a system reboot. The new binary starts with the old binary's state.
2. **The upgrade window is bounded.** The old service drains in-flight work, serialises state, and exits. The new service starts and registers. Clients retry. The entire sequence completes in milliseconds.
3. **Failure during upgrade is safe.** If the new service fails to start, the supervisor can restart the old service (or a fallback). The state memory objects are owned by the supervisor during the transition, so they are not lost.
4. **The building blocks are already proven.** Memory-object transfer, generation tracking, deterministic teardown, and bootstrap delivery are all exercised by the existing self-test suite. The live upgrade protocol adds approximately 150 lines of orchestration code.

17 Programming Model

This chapter describes the rules, patterns, and primitives for writing code that respects the async-first, shard-per-core model. It covers both kernel-side and userspace-side development.

17.1 Invariants

These are the rules that hold across all CharlotteOS code, kernel and userspace.

17.1.1 Only the owning LP mutates its state

Per-LP data is accessed only from the LP that owns it. Cross-LP mutation goes through message dispatch (`ShardMailbox<M>`, IPI closure, or IPI RPC), never through a shared lock acquired from a foreign LP.

17.1.2 Messages are owned values

Everything that crosses a shard boundary is an owned value — a typed `M` in a `ShardSender<M>`, an IPI closure, or a memory object. No shared references, no `Arc<Mutex<...>` as the normal programming model. `Send + 'static` boundaries enforce this at compile time.

17.1.3 References to shard-local state must not escape

`ShardLocal<T>::with(fn)` provides `&mut T` inside a closure. The closure must not store the reference, send it to another thread, or cross an `.await` point. The borrow is verified at runtime (owner-check plus re-entrancy flag).

17.1.4 Work stays on its assigned LP

A thread spawned on LP N stays on LP N. To submit work to another LP, use `try_run_on_lp(target, closure)` or `ShardSender::try_send`.

17.1.5 Observability returns owned snapshots

Never expose borrowed references into runtime internals. Diagnostic and introspection code captures state by value. Self-tests and the demo illustrate this pattern.

17.2 Kernel-side primitives

17.2.1 `PerLp<T>`

Lock-wrapped per-LP container. Use for data accessed from interrupt handlers, IPI handlers, or multiple LPs:

```
// Declaration:
static SCHEDULER: PerLp<RoundRobin> = PerLp::new();

// Access:
SCHEDULER.with(|scheduler| {
    scheduler.submit_ready_thread(tid);
});
```

The lock masks interrupts during the critical section on architectures that require it.

17.2.2 `ShardLocal<T>`

Lock-free per-LP container for single-LP, thread-only data:

```
// Declaration:
static MAILBOX: ShardLocal<ShardMailbox<Command>> = ShardLocal::new();

// Access (only from owning LP, not from interrupt context):
MAILBOX.with(|mb| {
    mb.try_send(Command::Ping)?;
});
```

Panics on wrong-LP access or re-entrant access.

17.2.3 `ShardMailbox<M>`

Typed bounded per-LP queue. Cloneable `ShardSender<M>` (any LP can send), single-consumer `ShardReceiver<M` (one per LP, accessed through `ShardLocal`):

```
let sender: ShardSender<Command> = mailbox.sender();
sender.try_send(Command::Process { data })?; // Returns Err(data) if full.
```

17.2.4 IPI closures

`try_run_on_lp(target_lp, || { ... })`. Boxes arbitrary work and delivers it to the target LP's IPI queue. Returns the closure on full queue (backpressure). Use for kernel-level cross-LP work where typed mailboxes are not appropriate (e.g., TLB shootdown, scheduler commands).

17.2.5 Completion API

`submit`, `submit_worker`, `complete`, `poll`, `wait`, `cancel`, `close`. The exit-observer path (`submit_worker`) is the intended default for fire-and-forget async kernel work:

```
let cap = submit_worker(|| {
    // Worker thread: do async work, then exit.
    // Completion fires automatically on exit.
});
let result = wait(cap)?; // Block until worker exits.
```

17.2.6 CQ ring

`CompletionQueueRing` for zero-syscall completion draining. Kernel side: `write(cap, result)`. Consumer side: `read() -> Option<CqEntry>`.

17.3 Userspace programming model

17.3.1 The launch contract

A CharlotteOS userspace program is a `no_std` ELF binary. It uses the `catten-rt` crate (the CRT, or C runtime) and the `entry!` macro:

```
#![no_std]
#![no_main]

use catten_rt::entry;

entry!(main);

fn main(ctx: Context) -> ! {
    // ctx provides:
    // - A connection to the name service
    // - The default CQ handle
    // - Heap configuration
    // - Bootstrap capabilities (device grants, etc.)

    let name_service = ctx.name_service();
    let nic_driver = name_service.lookup("driver.nic.v1")?;

    // ... service logic ...

    loop {
        // Poll tasks, drain CQ, wait for events.
    }
}
```

The program does not define `_start`, a panic handler, or an allocator. These are provided by `catten-rt`. The `entry!` macro wires them together, validates the launch ABI version, sets up the heap, and calls `main(ctx)`.

17.3.2 Bootstrap authority

The `Context` carries the domain's bootstrap capabilities:

- `name_service()` — a connection to the name service, for looking up other services.
- `default_cq()` — the CQ handle for submission and waiting.
- `device_grants()` — any `MmioRegion` or `Interrupt` capabilities granted to this domain.
- `heap_config()` — memory available for the allocator.

The `Context` is the only way a userspace program acquires capabilities at startup. There is no ambient authority, no global filesystem, no `/dev` to explore.

17.3.3 Caller identity is kernel authority

Syscalls derive the caller's address space from the trap frame. As a security boundary, userspace cannot name another domain's address space or capability table.

17.3.4 Backpressure is normal

Bounded queues and capability tables return `WouldBlock`. The correct response is to retry, yield, or propagate backpressure upstream. Do not assume an unbounded queue.

17.3.5 Sitas integration

When writing code that uses sitas on CharlotteOS:

- A sitas shard is an LP-affine CharlotteOS user thread.
- `CharlotteReactor::new(lp_id)` binds a reactor to the calling shard's LP.
- Async operations return kernel-owned completion handles. A completion handle is opaque: wait on it, poll it, close it, or pass it through a typed channel.
- Cross-shard work is an owned message. Prefer sitas channels and futures over shared mutable state.
- The executor parks on `CQ_WAIT` (via `ShardParker`), not on a busy-loop.

17.4 Rule-of-thumb guidance

- **Choose the right container:** `PerLp<T>` for interrupt-accessed or cross-LP data; `ShardLocal<T>` for single-LP, thread-only data.
- **Choose the right channel:** `ShardMailbox<M>` for typed cross-LP communication within the kernel. `ShardSender/Receiver` for typed cross-shard communication within userspace. Endpoint IPC for cross-domain communication.

- **Use `submit_worker` for fire-and-forget async work.** The worker just returns; the exit-observer completes the capability.
- **Register a CQ ring at address-space creation** so completions are visible to userspace without per-entry syscalls.
- **Accept backpressure.** `WouldBlock` or `Err(m)` means the receiver is slow, not that the system is broken. Retry or propagate upstream.
- **Buffers have one owner.** While an async operation owns a buffer, userspace must not mutate or free it. Cancellation still ends in a terminal completion.
- **Never accept ASID or page-table roots from userspace.** Caller identity comes from the trap frame. This is the security invariant.
- **Do not smuggle shared kernel state into userspace.** Shared pages (CQ ring, config page) are data queues with narrow layouts, not references to kernel objects.

17.5 The syscall ABI

The userspace–kernel boundary exposes these operations:

`submit(op_code, buffer) -> OperationId` Start an async operation. Returns immediately. Buffer ownership transfers to the kernel.
`wait(cq_id) -> Vec<CompletionEntry>` Block until a completion arrives on the given CQ. Returns all pending entries.
`poll(op_id) -> Option<Completed>` Non-blocking check. Returns `None` while in flight.
`cancel(op_id) -> CancelState` Request cancellation. May complete asynchronously.
`close(op_id)` Release an operation ID after its terminal completion was observed.

For IPC:

```
endpoint_create(interface, capacity) -> EndpointCap
connection_mint(endpoint, rights) -> ConnectionCap
scalar_send(connection, opcode, arg)
scalar_call(connection, opcode, arg) -> ReplyToken
memory_send(connection, opcode, attachments)
reply(token, result)
receive(endpoint) -> Message
```

The full ABI is described in `docs/async-syscall-abi.md` and implemented in `crates/catten/src/sysca`

17.6 Why the programming model matters for critical software

1. **Invariants are explicit and enforced.** That "Only the owning LP mutates its state" is enforced by `ShardLocal<T>` (wrong-LP access panics) and `ShardSender<M>` (ownership transfer at compile time). You cannot accidentally violate the shard discipline.
2. **The launch contract is minimal.** A userspace program receives exactly what it needs and nothing more. There is no global state, no inherited file descriptors, no ambient filesystem. The bootstrap capabilities are the program's entire universe of authority.

3. **Messages, not shared memory, are the default.** The safe, performant path is to send an owned value through a bounded channel. Shared memory with locks is the exception and not the rule. This reduces the number of places where concurrency bugs can hide.
4. **The ABI is auditable.** 49 syscalls across completion, IPC, memory, device, and CQ operations. Each has defined semantics, fixed-width arguments, and explicit error returns.
5. **The programming model is testable.** Every invariant (owner-check, re-entrancy guard, backpressure, cancellation contract) has a corresponding self-test. The test suite exercises the rules.

18 Development Workflow

This chapter explains how to build, boot, and test CharlotteOS. It is a practical guide for developers who want to work on the kernel, write userspace services, or run the self-tests.

18.1 Prerequisites

- **Rust nightly** (2026-07-18 or later), installed via `rustup`. The `rust-toolchain.toml` pins the exact version.
- **QEMU** for AArch64 (`qemu-system-aarch64`) or x86-64 (`qemu-system-x86_64`). On macOS, Apple Hypervisor Framework (HVF) acceleration is used automatically. On Linux, KVM is required for virtio-net runtime validation (HVF does not emulate EL0 MMIO instructions).
- **Build dependencies:** `cargo`, `just` (optional, for convenience targets in the Justfile).

18.2 Build targets

CharlotteOS supports three architectures via custom target JSON specs in `target_specs/`:

Target spec	Status
`aarch64-unknown-none-catten.json`	Primary development target; all features and self-tests pass
`x86_64-unknown-none-catten.json`	Boots, core self-tests pass; ring-3 userspace implemented but not
`riscv64gc-unknown-none-catten.json`	Planned, not actively maintained

18.2.1 Kernel build

```
# AArch64 (headless, serial console only):
cargo build --package catten \
  --target target_specs/aarch64-unknown-none-catten.json \
  --no-default-features --features acpi

# x86-64:
cargo build --package catten \
  --target target_specs/x86_64-unknown-none-catten.json \
  --no-default-features --features acpi
```

The `display` feature requires a C library for the flanterm framebuffer terminal. Headless boot uses serial output only (AArch64: PL011 UART; x86-64: 16550 COM1).

18.2.2 Userspace service programs

Service binaries are built separately because they target `no_std` / `no_main` with custom EL0 targets that depend on build-std:

```
# EL0 service programs:
cargo build --release --package catten-services \
  --target aarch64-unknown-none.json \
  -Zbuild-std=core,alloc

# Sitas user binary (requires external sitas crates):
cargo +nightly build --package catten-user \
  --target aarch64-unknown-none.json \
  -Zbuild-std=core,alloc
```

18.3 Booting in QEMU

18.3.1 AArch64 (macOS, HVF)

```
# Single-core boot:
./scripts/boot-smp1.sh

# Dual-core boot:
./scripts/boot-smp2.sh

# Or specify timeout and core count directly:
./scripts/qemu-boot-macos.sh 60 2
```

18.3.2 x86-64

```
# Requires -cpu max for RDTSCP/FSGSBASE support:
./scripts/boot-x86.sh
```

18.3.3 Expected output

```
CharlotteOS booting...
[init] BSP initialization complete.
[init] Secondary CPUs brought online.
[init] Starting scheduler...
[async-demo] coordinator: submitted worker, now awaiting completion...
[async-demo] worker: performing async work (sleep 50ms)
[async-demo] worker: work finished, exiting (completion fires on exit)
[async-demo] coordinator: COMPLETION RECEIVED, result Ok(42)
[async-demo] SUCCESS: async syscall round-trip complete
Testing Complete. All Tests Passed!
```

18.4 Self-tests

There is no host-side `cargo test`. All tests are **boot-time kernel self-tests** — whitebox integration tests run from `self_test::run_self_tests()` called after `INIT_BARRIER.wait()`. Each test uses

`assert!()` / `assert_eq!()`; panic equals failure.

Current test suites (both architectures pass on AArch64; core tests pass on x86-64):

- `completion.rs` — buffer ownership, cancel/deferred-reclaim, backpressure.
- `cq.rs` — CQ ring write/drain/overflow.
- `cq_completion.rs` — CQ ring integrated with completion subsystem.
- `syscall.rs` — dispatch table routing via synthetic TrapFrame.
- `ipi.rs` — bounded queue, backpressure, closure dispatch.
- `shard.rs` — `ShardLocal<T>` owner-check / re-entrancy guard, `ShardMailbox<M>` send/recv.
- `el0.rs` — real-EL0 user thread (AArch64 only).
- `el0_demo.rs` — EL0 userspace smoke test.
- `el0_ipc.rs` — EL0 endpoint IPC between domains.
- `el0_net.rs` — EL0 networking test.
- `el0_pingpong.rs` — bidirectional EL0 IPC.
- `el0_raft.rs` — two-node Raft election over local IPC.
- `el0_service.rs` — EL0 service lifecycle (spawn, IPC, teardown).
- `el0_uart.rs` — userspace UART driver (delegated MMIO + IRQ, crash recovery).
- `el0_sitas.rs` — sitas executor integration.
- `scheduler_lifecycle.rs` — spawn, block, wake, abort, migration-safe migration, timer-wait affinity.
- `ipc.rs` — IPC path tests (scalar, memory, delegation, reply tokens).
- `memory/allocator.rs` — heap allocator, stack allocator.
- `memory/object.rs` — memory object lifecycle tests.
- `adversarial.rs` — cancellation races, misuse paths, error-path testing.

18.5 The async-syscall demo

`crates/catten/src/demo.rs` is the canonical end-to-end test of the async model. It:

1. Spawns a coordinator and worker from `bsp_main`.
2. The coordinator calls `submit_worker`, which spawns a worker thread and returns a capability that fires when the worker exits.
3. The worker sleeps 50ms (exercising the timer and block-yield-wake cycle).
4. The worker exits; the exit-observer fires; the capability completes.
5. The coordinator (blocked on `wait(cap)`) wakes and reads the result.

This exercises: thread creation, scheduling, timer events, the block-yield-wake cycle, the exit-observer completion path, and the wait/poll ABI. If this demo passes, the core async machinery is working.

18.6 CI pipeline

GitHub Actions (`.github/workflows/ci.yml`):

1. Builds both architectures with `-D warnings` (warnings are errors).
2. Runs formatting checks (`cargo fmt -check`).
3. Boots the AArch64 QEMU target and greps for "Testing Complete. All Tests Passed!".
4. Builds the kernel for x86-64.

The CI gate ensures the `dev` branch always compiles cleanly and passes boot tests.

18.7 Codebase reading guide

For new contributors, the recommended reading order:

1. `crates/catten/src/main.rs` — kernel entry point, init sequence.
2. `crates/catten/src/completion/mod.rs` — the completion subsystem: capability table, submit/complete/poll/wait/cancel/close, `submit_worker`, exit-observer.
3. `crates/catten/src/completion/cq.rs` — the CQ ring buffer.
4. `crates/catten/src/syscall/mod.rs` — syscall dispatch table.
5. `crates/catten/src/demo.rs` — the end-to-end async demo.
6. `crates/catten/src/cpu/scheduler/` — threads, wakers, `RoundRobin`, `block_thread`.
7. `crates/catten/src/cpu/multiprocessor/` — SMP: IPI, `PerLp<T>`, `ShardLocal<T>`, shard mailboxes.
8. `crates/catten/src/cpu/isa/` — ISA abstraction layer. Start with `interface/` (traits), then `aarch64/` (reference implementation).
9. `crates/catten/src/service/` — supervisor, ELF loader, bootstrap.
10. `crates/catten/src/device_management/` — device driver framework.
11. `crates/catten-services/src/bin/` — reference EL0 services.
12. `crates/catten-rt/src/lib.rs` — userspace runtime.

18.8 Troubleshooting

Triple fault during boot Check that the panic handler is working. Early faults before heap initialization rely on `early_logln!` (direct serial writes). Ensure the serial port is configured correctly.

Thread hangs after blocking The block-yield-wake cycle requires that `block_thread` not clear `current_handle` before `yield_lp` saves the thread's context. Verify that the blocked thread is still the current thread at the point of the yield.

ASID-0 collision The first user address space must not receive ASID 0 (which equals `KERNEL_ASID`). The address-space table must be pre-populated with the kernel AS at index 0.

x86-64 LA57 mismatch Derive the address width from `CR4.LA57` (the active paging mode), not from CPUID capability. Limine boots with 4-level paging even on CPUs that support 5-level.

A Glossary

AS (Address Space) A page-table context identified by an `AddressSpaceId` (0 = kernel, >0 = user). Contains the `TTBR0_EL1` (user) and `TTBR1_EL1` (kernel) pointers on AArch64, or `CR3` on x86-64.

Block (verb) To remove a running thread from the scheduler and register its `Waker` on an `Observable`. The thread yields; when the event fires the waker re-admits it.

Capability An unforgeable per-domain handle that names a kernel object and carries rights. Answers: "May I perform this operation?"

Capability table A per-AS sparse table mapping capability indices to kernel objects. Used for the thread table, address-space table, and per-domain capability table.

Completion A kernel object representing an in-flight or completed async operation. `Observable`: threads `wait` on it, `complete` signals it.

Connection capability A client-side authority to send or call an endpoint. Minted from an endpoint, typically by the name service.

CQ ring (Completion Queue) A shared-memory ring buffer for zero-syscall completion delivery from the kernel to userspace.

EL0 Exception Level 0 (AArch64). Userspace execution.

EL1 Exception Level 1 (AArch64). Kernel execution.

Endpoint A bounded server-side IPC receive object. Has an owning domain, a queue capacity, and a lifecycle.

Exit-observer An `Observer` registered on a `Thread`; fires when the thread is dropped (exits). The ABI's intended completion mechanism: `submit_worker` registers the capability as the worker's exit-observer.

HHDM (Higher Half Direct Mapping) A linear mapping of all physical memory into the kernel's virtual address space, provided by the bootloader (Limine).

IdTable A sparse `Vec<Option<T>>` with ID recycling. Used for the thread table, address-space table, and capability table.

IOMMU / SMMU I/O Memory Management Unit. Hardware that enforces DMA access permissions from devices, analogous to the MMU for CPU access.

IPI (Inter-Processor Interrupt) A signal from one LP to another. The kernel's IPI queues are bounded per-LP queues carrying typed commands.

LP (Logical Processor) One hardware core (or hyperthread). The unit of sharding in both the kernel and sitas.

Memory object A page-backed kernel object with explicit ownership and capability semantics. Can be mapped, moved, lent, or copied between domains.

MMU Memory Management Unit. Hardware that enforces page-table permissions for CPU access.

Name service A userspace service that maps human-readable names to connection capabilities. The kernel does not understand service names.

Notification "State may have changed; inspect the corresponding object or queue." May be coalesced. Is not itself a result.

Observable / Observer The event-notification pattern. An `Observable` holds `Weak<dyn Observer>` references; when it fires, it calls `Observer::notify()`.

PerLp<T> A boxed slice of per-LP cells with lock wrapping. Interrupt-safe (masks interrupts on acquire). Use for data accessed from interrupt handlers or multiple LPs.

Protection domain An address space and authority boundary. Owns a capability table, memory mappings, threads, and failure state. Typically implemented as a process.

Quantum timer The per-LP periodic timer (10 ms default) that drives preemptive context switching in the

`RoundRobin` scheduler.

Reaper The deferred-thread-cleanup mechanism: dead threads are moved to `DEAD_THREADS` and dropped from the next thread's context after the context switch.

Reply token One-shot authority to complete a blocking or deferred endpoint call. Created by the kernel, bound to the caller's pending call, consumable exactly once, invalidated by cancellation or teardown.

Shard A userspace ownership and execution domain. One executor runs its tasks; at most one task at a time mutates shard-owned state; cross-shard access goes through typed messages.

ShardLocal<T> A lock-free per-LP container using `UnsafeCell<T>` with owner-check and borrow-flag guard. sitas-derived. Use for single-LP, thread-only kernel data.

ShardMailbox<M> A typed bounded per-LP queue with `ShardSender<M>` (cloneable, any LP can send) and `ShardReceiver<M>` (single-consumer). First-class backpressure.

Supervisor Kernel-side component that creates and destroys protection domains, loads ELF images, delivers bootstrap capabilities, and handles domain teardown.

TrapFrame An architecture-specific snapshot of the user register state at the moment an exception was taken. Provides the caller's ASID for syscall dispatch.

Waker An `Observer` wrapping a `ThreadId`. When notified, calls `submit_ready_thread(tid)`. In userspace, a Rust `Waker` is a hint to repoll a future — it is not a kernel completion.

Yield To voluntarily give up the LP by calling `yield_lp()`. Saves the thread's context and lets the scheduler run the next ready thread.

B Implementation Status

This appendix catalogs what has been built and verified, organized by architectural phase. It is a snapshot; the repository history is the authoritative record.

B.1 Implementation phases

Phase	Description	Status
1	Capability table, rights masks, object teardown	Done
2	Endpoint IPC v1: scalar send/call/receive/reply, connection delegation, reply tokens	Done
3	Userspace name/service manager, bootstrap delivery, ELF loader, supervisor, generation tracking	Done
4	First-class memory objects: allocate, map, unmap, close, ownership accounting	Done
5	Memory IPC: Copy, Move, BorrowRead, BorrowWrite, reply-bound revocation, cancellation, server-death recovery	Done
6	CQ subsystem normalization: operation IDs, detached submission, CQ_WAIT/CQ_WAKE, per-shard CQ partitioning, backlog batching, 32-byte completion records	Done
7	Sitas endpoint/CQ backend: endpoint readiness binding, unified shard wait, ShardExecutor (budgeted polling), ShardParker seam (spin free), per-shard CQ rings	Done
8	Userspace UART driver: delegated MMIO + IRQ, EL0 MMIO writes, interrupt-driven deferred reads, crash recovery	Done
9	Virtio-net driver: PCI discovery, BAR0 + IRQ delegation, virtio init, MAC read, virtqueue setup, frame TX/RX (compiles). Smoltcp 0.13 adapter + TCP/IP service binary (compile). Runtime validation pending on KVM.	Driver built, not runtime-validated
10	Lock-free device interrupt delivery (deferred wake), idempotent scheduler wakes, sustained-window load rebalancing	Done
11	Userspace startup ABI: ELF `_start`, `entry!` macro, typed `Context`, fixed-width versioned launch header, explicit startup input	Done

B.2 Success criteria

1. Two EL0 domains communicate through a bounded endpoint — **Verified.**
2. Deferred async call while a shard continues other tasks — **Verified.**
3. Moved memory object — sender locked out until return — **Verified.**
4. Lending enforced by mappings, including death cleanup — **Verified.**
5. Single `CQ_WAIT` for endpoint + completions + device interrupts — **Verified.**
6. Terminal CQ results survive ring overflow (non-lossy backlog) — **Verified.**
7. Coalesced remote shard wakes (idempotent scheduler fix) — **Verified.**
8. Userspace UART driver with only delegated MMIO + IRQ — **Verified.**
9. Restart invalidates stale connections + device reset + op reconciliation — **Verified.**
10. Virtio-net data path with batched packet buffers — **Compiled, runtime pending.**
11. Fixed-width stable ABI representations (32-byte CQ entries) — **Implemented; not formally audited.**
12. Capability misuse and cancellation race adversarial tests — **Partial; six IPC error-path tests.**

B.3 Verified (boots on QEMU, zero panics)

- **AArch64:** boots, all self-tests pass, the full EL0 service suite (name service, echo, client, UART driver, sitas executor) runs through `crt0/cmain`. The sitas `basic_kv` binary runs with 2 shards on per-shard CQ rings.
- **x86-64:** boots with `-cpu max`, core self-tests pass. Ring-3 userspace is implemented but not yet exercised.
- **Completion subsystem:** submit, complete, poll, wait, cancel, close, `submit_worker` (exit-observer), CQ ring, CQ ring integrated with AS, non-lossy CQ overflow backlog, per-shard CQ partitioning, 32-byte entries.
- **Endpoint IPC:** bounded server queues, scalar messages, connection delegation with rights attenuation, memory-object attachments (move/lend/copy), deferred replies, reply-time connection delegation.
- **Device capabilities:** MmioRegion and Interrupt object types, user-accessible Device-nGnRnE page mapping, GICv3 SPI enable/route/mask, IRQ-to-CQ coalesced delivery (lock-free deferred wake).
- **Userspace drivers:** reference UART driver (PL011) with delegated MMIO + IRQ, interrupt-driven deferred reads, crash-test restart with device reset and operation reconciliation.
- **Syscall dispatch:** AArch64 SVC handler dispatches 49 syscalls. Caller ASID is derived from the trap frame, never a userspace parameter.
- **Scheduler:** RoundRobin, quantum timer, block/yield/wake, idempotent wakes, deferred reaper, per-LP thread pinning.
- **Sitas integration:** sitas-core compiles for aarch64-none. sitas-charlotte links against the kernel's syscall ABI. Per-shard CQ rings mapped by the loader contract. ShardParker seam retires busy-waiting.
- **Protocol crates:** `charlotte-protocol-net`, `charlotte-protocol-msg`.
- **Networking:** smoltcp 0.13 adapter and TCP/IP service binary compile for aarch64-none.

- **Raft:** two-node election over local endpoint IPC on 4-LP QEMU/HVF. Timer-driven elections via completion events. Verifier domains terminate rather than polling.
- **CI:** GitHub Actions builds both architectures with `-D warnings`, QEMU boot gate on AArch64.

B.4 Known limitations

- **x86-64 ring-3 userspace:** SYSCALL entry trampoline exists; the EL0 test and real-EL0 user thread are AArch64-only.
- **Virtio-net runtime:** the driver and TCP/IP stack compile but cannot be runtime-validated on macOS (HVF does not emulate EL0 MMIO instructions). Validation requires a Linux KVM host.
- **General process loader:** the ELF loader is fixed-address for reference service images, not a general loader with argv/env/auxv-style setup.
- **Resource accounting:** per-domain quotas are not yet implemented; capability-table capacity is fixed.
- **Cross-machine Raft:** blocked on NIC runtime validation (KVM) and the reliable message layer implementation.
- **IOMMU/SMMU integration:** designed but not implemented.

C Lessons Learned

Bootstrapping an async-first kernel on real hardware (QEMU) surfaced bugs that static analysis and self-tests never caught. This appendix records the most instructive, and the design insights they produced.

C.1 Bugs found on hardware

C.1.1 Panic handler recursed through heap-dependent logging

Symptom: any fault during early boot silently triple-faulted instead of printing a panic message. **Root cause:** the panic handler used the normal logging path, which allocated from the kernel heap. A fault before the heap was ready caused the panic handler to fault, re-entering itself, recursing until stack overflow and triple fault. **Fix:** the panic handler now uses `early_logln!` (direct serial writes, no heap dependency). This unmasked every subsequent bug.

C.1.2 Blocked threads lost their execution context

Symptom: a thread that blocked in `wait()` restarted from its entry point when woken. **Root cause:** `block_thread` cleared the scheduler's `current_handle` before `yield_lp` saved the thread's context, so the save was skipped. **Fix:** a self-blocking thread stays `current_handle` until the yield saves its context.

C.1.3 RwLock held-across-call deadlocks

Symptom: `sleep()` worked with logging but hung without it (a heisenbug). **Root cause:** an `if let` binding extended an RwLock read guard's lifetime across a call that tried to acquire the write lock. **Fix:** bind the value out of the scrutinee first, so the temporary guard drops before the re-locking call.

C.1.4 Quantum timer grew without bound

Symptom: the per-LP timer queue contained thousands of quantum events after extended operation. **Root cause:** manual yields enqueued a new quantum event each time. **Fix:** an `armed` flag keeps exactly one quantum event in flight.

C.1.5 A thread cannot free its own kernel stack

Symptom: a returning kernel thread hung during teardown. **Root cause:** `ThreadContext::Drop` deallocated the stack while the thread was still executing on it. **Fix:** a deferred reaper moves dead threads to a zombie list and drops them from the next thread's context after the switch.

C.1.6 LA57 paging-mode mismatch

Symptom: a `#GP` with a non-canonical address on x86-64. **Root cause:** the address width was derived from CPUID (57-bit capability) instead of `CR4.LA57` (the active mode, 48-bit under Limine). **Fix:** derive from the active paging mode.

C.1.7 ASID-0 collision

Symptom: the first user thread faulted at its entry point. **Root cause:** the first user address space received ASID 0, which equals `KERNEL_ASID`, so the thread was created as a kernel thread (EL1), where PXN forbade execution of user code. **Fix:** pre-populate the table with the kernel AS at index 0.

C.1.8 x86-64 trampoline halted on thread return

Symptom: the async demo's worker completed but the coordinator never resumed. **Root cause:** the x86-64 trampoline entered a `hlt` loop when a thread returned, instead of calling `abort()`. **Fix:** make the x86-64 trampoline call `abort` on return.

C.2 Key insights

1. **The block-yield-wake cycle is the hard problem.** Designing the completion ABI was straightforward compared to making `wait()` correctly save and restore thread state through the cooperative scheduler. Every scheduling primitive exercised a new path and surfaced a new bug.
2. **Booting on hardware finds bugs that static analysis cannot.** The panic-handler recursion, ASID-0 collision, and LA57 paging mismatch were invisible in self-tests and logical reasoning. Every one was found by observing actual fault register values.
3. **The observer/waker pattern is the right primitive.** Once `Completion` was made `Observable` and `wait()` used the same path as `sleep()`, the entire completion subsystem fell into place. The exit-observer mechanism is the natural endpoint: a thread exiting *is* the completion event.
4. **The async model genuinely removes the retrofit tax.** On CharlotteOS, `wait(cap)` blocks the same way `sleep(50ms)` blocks — both register a `Waker` on an `Observable` and yield. There is no emulation, no thread pool, no signal-handler tightrope. The kernel and the runtime agree about what is fundamental.
5. **The sitas traits were essential for co-design.** The `ReactorBackend` trait became the executable specification for the async syscall ABI. The sitas-charlotte backend validates ABI shapes before any kernel path exists. This bidirectional validation is the core of the collaboration.
6. **The deferred reaper is necessary for any thread that exits.** A thread cannot free its own stack. The reaper pattern is the minimum safe discipline.
7. **`RwLock` held-across-call is a systemic hazard.** The Rust pattern of `if let Some(val) = lock().method()` extends the guard's lifetime across the body. Bind the value first, then use it.

D Architectural Redirections

This appendix records the explicit design decisions that changed the project's direction from its earlier experiments. It is drawn from the architecture document `docs/charlotte-sitas-xous-architecture.md` and reflects the transition from an earlier "universal completion-capability" model to the three-primitive architecture described in this manual.

D.1 What to retain from the current implementation

The following work aligns with the architecture and should continue:

- AArch64 EL0/SVC path.
- Caller identity derived from the current trap/thread context.
- Per-address-space capability tables.
- Shared CQ ring.
- Non-lossy kernel CQ backlog.
- `CQ_WAIT` blocking through scheduler/observer machinery.
- Bounded queues and explicit `WouldBlock`.
- No-std sitas split and CharlotteOS backend.
- Per-shard executor model.
- Typed in-kernel `ShardMailbox<M>`.
- Closure-based `ShardLocal<T>` with restricted use.
- Segment-aware ELF loading.
- QEMU boot CI across AArch64 and x86-64.

D.2 What to reframe

These components are correct but their *role* in the architecture has shifted:

- **Completion capability work** is the **operation completion subsystem**, not the universal service IPC subsystem. Use completion queues for kernel and device operations; use endpoints for cross-domain service calls.
- **Observer/waker code** is internal scheduling machinery, not the public semantic identity of all async objects. A Waker is a hint to repoll a future; it is not a completion record.
- **IPI queue closures** are a temporary kernel work mechanism, not the long-term cross-domain messaging model. Use typed mailboxes or closed kernel enums for subsystem-specific work.
- **The earlier mailbox syscall ABI** is a smoke-test interface only. The stable ABI is endpoint IPC.

D.3 What to replace

D.3.1 Stop expanding raw mailbox syscalls

A mailbox tied directly to LP identity is not a complete userspace service abstraction: it exposes placement where authority should be exposed, lacks protocol identity, conflates shard transport with domain IPC, and lacks move/lend semantics. **Direction:** implement endpoint and connection capabilities independent of LP identity. Clients address the service connection, not a target LP.

D.3.2 Do not allocate one completion capability per service request

For userspace service calls, the natural object is a reply token. For high-rate kernel operations, per-operation capabilities create unnecessary table pressure. **Direction:** use reply tokens for endpoint calls; operation IDs plus CQ entries for ordinary kernel operations; first-class operation capabilities only where independent authority is required.

D.3.3 Separate readiness, notification, and completion

Their contracts differ: readiness means progress may be possible; notification means inspect state; completion means an operation changed state and has result data. Keep one aggregated wait mechanism, but define distinct object semantics.

D.3.4 Replace arbitrary cross-LP closures

Boxed `FnOnce()` work transported through the IPI layer is hard to account, difficult to inspect, and brittle under backpressure. **Direction:** use closed kernel enums for mandatory kernel RPCs and typed mailbox instances for subsystem-specific work.

D.3.5 Do not equate shard wake with IPI delivery

Remote wake mapping directly to `send_ipi(target_lp)` can generate excessive interrupts. **Direction:** queue first, mark pending, send a coalesced reschedule IPI only when the target cannot otherwise observe the transition.

D.3.6 Introduce memory objects before rich IPC payloads

Xous's most important contribution — move/lend semantics — cannot be implemented safely with transient pointers. **Direction:** prioritize memory-object capability, mapping/unmapping, move between domains, temporary lending, revocation on reply/cancel/death, and optional DMA integration. Only then promote rich IPC payloads.

D.3.7 Treat service discovery as userspace policy

Do not add string-based global service lookup to the kernel. Build a userspace name/policy service that receives bootstrap authority and delegates connection capabilities.